

Task 1

The Data Engineering team needs a summary of employee records loaded into the Data Warehouse. Display: • Department • Total Employees • Average Salary • Highest Salary • Lowest Salary Sort by Average Salary in descending order.

```
CREATE TABLE Employee (  
  EmpID INT PRIMARY KEY,  
  EmpName VARCHAR(15),  
  Salary INT,  
  Department VARCHAR(15),  
  Age INT  
);  
INSERT INTO Employee  
(EmpID, EmpName, Salary, Department, Age)  
VALUES  
(101, 'Alice', 55000, 'HR', 28),  
(102, 'Bob', 62000, 'IT', 32),  
(103, 'Charlie', 48000, 'Finance', 26),  
(104, 'David', 70000, 'IT', 35),  
(105, 'Eva', 53000, 'Marketing', 29),  
(106, 'Frank', 60000, 'Sales', 31),  
(107, 'Grace', 75000, 'Finance', 38),  
(108, 'Henry', 45000, 'HR', 25),  
(109, 'Ivy', 68000, 'Marketing', 34),  
(110, 'Jack', 52000, 'Sales', 27);  
  
SELECT  
  Department,  
  COUNT(*) AS Total_Employees,  
  AVG(Salary) AS Average_Salary,  
  MAX(Salary) AS Highest_Salary,  
  MIN(Salary) AS Lowest_Salary  
FROM Employee  
GROUP BY Department  
ORDER BY Average_Salary DESC;
```

Task 2

Find departments whose total salary is greater than the company average department salary. Display: •
Department • Total Salary

```
Create table Employee (  
    EmpID int primary key,  
    EmpName Varchar(15),  
    Salary int,  
    Department varchar(15),  
    Age int  
);  
insert into Employee (EmpID, EmpName, Salary, Department, Age)  
values(101, 'Alice', 55000, 'HR',28),  
(102, 'Bob', 62000, 'IT', 32),  
(103, 'Charlie', 48000, 'Finance', 26),  
(104, 'David', 70000, 'IT', 35),  
(105, 'Eva', 53000, 'Marketing', 29),  
(106, 'Frank', 60000, 'Sales', 31),  
(107, 'Grace', 75000, 'Finance', 38),  
(108, 'Henry', 45000, 'HR', 25),  
(109, 'Ivy', 68000, 'Marketing', 34),  
(110, 'Jack', 52000, 'Sales', 27);  
  
WITH DepartmentTotals AS (  
    SELECT  
        Department,  
        SUM(Salary) AS Total_Salary  
    FROM Employee  
    GROUP BY Department  
)  
SELECT  
    Department,  
    Total_Salary  
FROM DepartmentTotals  
WHERE Total_Salary > (  
    SELECT AVG(Total_Salary)  
    FROM DepartmentTotals  
)  
ORDER BY Total_Salary DESC;
```

Task 3

Identify employees whose salary is higher than the average salary of their department. Display: • Employee Name • Department • Salary • Department Average

```
Create table Employee (
    EmpID int primary key,
    EmpName Varchar(15),
    Salary int,
    Department varchar(15),
    Age int
);
insert into Employee (EmpID, EmpName, Salary, Department, Age)
values(101, 'Alice', 55000, 'HR',28),
(102, 'Bob', 62000, 'IT', 32),
(103, 'Charlie', 48000, 'Finance', 26),
(104, 'David', 70000, 'IT', 35),
(105, 'Eva', 53000, 'Marketing', 29),
(106, 'Frank', 60000, 'Sales', 31),
(107, 'Grace', 75000, 'Finance', 38),
(108, 'Henry', 45000, 'HR', 25),
(109, 'Ivy', 68000, 'Marketing', 34),
(110, 'Jack', 52000, 'Sales', 27);

WITH DepartmentAverages AS (
    SELECT
        Department,
        AVG(Salary) AS Department_Average
    FROM Employee
    GROUP BY Department
)
SELECT
    e.EmpName,
    e.Department,
    e.Salary,
    da.Department_Average
FROM Employee AS e
JOIN DepartmentAverages AS da
    ON da.Department = e.Department
WHERE e.Salary > da.Department_Average
ORDER BY e.Department, e.Salary DESC;
```

Task 4

Rank departments based on total salary expenditure. Display: • Department • Total Salary • Rank

```
Create table Employee (  
    EmpID int primary key,  
    EmpName Varchar(15),  
    Salary int,  
    Department varchar(15),  
    Age int  
);  
insert into Employee (EmpID, EmpName, Salary, Department, Age)  
values(101, 'Alice', 55000, 'HR',28),  
(102, 'Bob', 62000, 'IT', 32),  
(103, 'Charlie', 48000, 'Finance', 26),  
(104, 'David', 70000, 'IT', 35),  
(105, 'Eva', 53000, 'Marketing', 29),  
(106, 'Frank', 60000, 'Sales', 31),  
(107, 'Grace', 75000, 'Finance', 38),  
(108, 'Henry', 45000, 'HR', 25),  
(109, 'Ivy', 68000, 'Marketing', 34),  
(110, 'Jack', 52000, 'Sales', 27);  
  
WITH DepartmentTotals AS (  
    SELECT  
        Department,  
        SUM(Salary) AS Total_Salary  
    FROM Employee  
    GROUP BY Department  
)  
SELECT  
    Department,  
    Total_Salary,  
    DENSE_RANK() OVER (  
        ORDER BY Total_Salary DESC  
    ) AS Salary_Rank  
FROM DepartmentTotals  
ORDER BY Salary_Rank;
```

Task 5

Find duplicate Employee IDs generated during ETL loading.

```
Create table Employee (  
    EmpID int,  
    EmpName Varchar(15),  
    Salary int,  
    Department varchar(15),  
    Age int  
);  
insert into Employee (EmpID, EmpName, Salary, Department, Age)  
values(101, 'Alice', 55000, 'HR',28),  
(102, 'Bob', 62000, 'IT', 32),  
(103, 'Charlie', 48000, 'Finance', 26),  
(104, 'David', 70000, 'IT', 35),  
(105, 'Eva', 53000, 'Marketing', 29),  
(106, 'Frank', 60000, 'Sales', 31),  
(107, 'Grace', 75000, 'Finance', 38),  
(108, 'Henry', 45000, 'HR', 25),  
(109, 'Ivy', 68000, 'Marketing', 34),  
(110, 'Jack', 52000, 'Sales', 27);  
  
SELECT  
    EmpID,  
    COUNT(*) AS Duplicate_Count  
FROM Employee  
GROUP BY EmpID  
HAVING COUNT(*) > 1  
ORDER BY EmpID;
```

Task 6

Display employees whose Department_ID has no matching record in the Department table.

```
Create table Employee (  
    EmpID int primary key,  
    EmpName Varchar(15),  
    Salary int,  
    Age int,  
    departmentID int  
);  
insert into Employee (EmpID, EmpName, Salary, Age)  
values(101, 'Alice', 55000,28, 1),  
(102, 'Bob', 62000, 32,2),  
(103, 'Charlie', 48000, 26,3),  
(104, 'David', 70000, 35,2),  
(105, 'Eva', 53000, 29,4),  
(106, 'Frank', 60000, 31,5),  
(107, 'Grace', 75000, 38,3),  
(108, 'Henry', 45000, 25,1),  
(109, 'Ivy', 68000, 34,4),  
(110, 'Jack', 52000,27,7);  
Create table department (  
    DepartmentID int,  
    DepartmentName varchar(15)  
);  
insert into department(departmentID,departmentname)  
values (1,'HR'),  
(2, 'IT'),  
(3,'Finance'),  
(4,'Marketing'),  
(5,'sales');  
SELECT  
    e.EmpID,  
    e.EmpName,  
    e.DepartmentID  
FROM Employee AS e  
LEFT JOIN Department AS d  
    ON d.DepartmentID = e.DepartmentID  
WHERE d.DepartmentID IS NULL  
ORDER BY e.EmpID;
```

Task 7

Each employee may have multiple records due to historical loads. Display only the latest record for every employee.

```
Create table Employee (
  EmpID int primary key,
  EmpName Varchar(15),
  Salary int,
  Department varchar(15),
  Age int,
  Loaddate date
);
insert into Employee (EmpID, EmpName, Salary, Department, Age, Loaddate)
values(101, 'Alice', 55000, 'HR',28,7-7-2025),
(102, 'Bob', 62000, 'IT', 32,12-10-2019),
(103, 'Charlie', 48000, 'Finance', 26,8-10-2026),
(104, 'David', 70000, 'IT', 35, 06-10-2024),
(105, 'Eva', 53000, 'Marketing', 29,04-03-2022),
(106, 'Frank', 60000, 'Sales', 31,01-03-2021),
(107, 'Grace', 75000, 'Finance', 38,08-2-2018),
(108, 'Henry', 45000, 'HR', 2022,02-12-2018),
(109, 'Ivy', 68000, 'Marketing', 34,11-3-2023),
(110, 'Jack', 52000, 'Sales', 27, 07-05-2025);

WITH LatestRecords AS (
  SELECT
    e.*,
    ROW_NUMBER() OVER (
      PARTITION BY EmpID
      ORDER BY LoadDate DESC
    ) AS rn
  FROM Employee AS e
)
SELECT
  EmpID,
  EmpName,
  Salary,
  Department,
  Age,
  LoadDate
FROM LatestRecords
WHERE rn = 1
ORDER BY EmpID;
```

Task 8

Calculate the running total of employees hired month-wise.

```
Create table Employee (
  EmpID int primary key,
  EmpName Varchar(15),
  Salary int,
  Department varchar(15),
  Age int,
  joineddate date
);
insert into Employee (EmpID, EmpName, Salary, Department, Age, Loaddate)
values(101, 'Alice', 55000, 'HR',28,7-7-2025),
(102, 'Bob', 62000, 'IT', 32,12-10-2019),
(103, 'Charlie', 48000, 'Finance', 26,8-10-2026),
(104, 'David', 70000, 'IT', 35, 06-10-2024),
(105, 'Eva', 53000, 'Marketing', 29,04-03-2022),
(106, 'Frank', 60000, 'Sales', 31,01-03-2021),
(107, 'Grace', 75000, 'Finance', 38,08-2-2018),
(108, 'Henry', 45000, 'HR', 2022,02-12-2018),
(109, 'Ivy', 68000, 'Marketing', 34,11-3-2023),
(110, 'Jack', 52000, 'Sales', 27, 07-05-2025);

WITH MonthlyHires AS (
  SELECT
    YEAR(JoinedDate) AS Hire_Year,
    MONTH(JoinedDate) AS Hire_Month,
    MONTHNAME(JoinedDate) AS Month_Name,
    COUNT(*) AS Employees_Hired
  FROM Employee
  GROUP BY YEAR(JoinedDate), MONTH(JoinedDate), MONTHNAME(JoinedDate)
)
SELECT
  Hire_Year,
  Hire_Month,
  Month_Name,
  Employees_Hired,
  SUM(Employees_Hired) OVER (
    ORDER BY Hire_Year, Hire_Month
  ) AS Running_Total
FROM MonthlyHires
ORDER BY Hire_Year, Hire_Month;
```


Task 9

Display the top three highest-paid employees in every department.

```
Create table Employee (  
    EmpID int primary key,  
    EmpName Varchar(15),  
    Salary int,  
    Department varchar(15),  
    Age int  
);  
insert into Employee (EmpID, EmpName, Salary, Department, Age)  
values(101, 'Alice', 55000, 'HR',28),  
(102, 'Bob', 62000, 'IT', 32),  
(103, 'Charlie', 48000, 'Finance', 26),  
(104, 'David', 70000, 'IT', 35),  
(105, 'Eva', 53000, 'Marketing', 29),  
(106, 'Frank', 60000, 'Sales', 31),  
(107, 'Grace', 75000, 'Finance', 38),  
(108, 'Henry', 45000, 'HR', 25),  
(109, 'Ivy', 68000, 'Marketing', 34),  
(110, 'Jack', 52000, 'Sales', 27);  
  
WITH RankedEmployees AS (  
    SELECT  
        EmpID,  
        EmpName,  
        Department,  
        Salary,  
        ROW_NUMBER() OVER (  
            PARTITION BY Department  
            ORDER BY Salary DESC, EmpID  
        ) AS Salary_Rank  
    FROM Employee  
)  
SELECT  
    EmpID,  
    EmpName,  
    Department,  
    Salary  
FROM RankedEmployees  
WHERE Salary_Rank <= 3  
ORDER BY Department, Salary DESC;
```

Task 10

```
Create a View that displays active employee details along with department information. */
create table Employee (EmpID int ,departmentID int primary key,EmpName varchar(50) ,Salary decimal(10,2),HireDate date,
Statu varchar(20));
insert into Employee values(101,1, 'Smith', 'IT', 65000.00, '2021-03-15', 'Active'),
(102,2, 'Johnson', 'HR', 55000.00, '2020-07-10', 'Active'),
(103,3, 'Brown', 'Finance', 60000.00, '2019-11-25', 'Inactive'),
(104, 4, 'Williams', 'Marketing', 52000.00, '2022-01-05', 'Active'),
(105,1, 'David', 'IT', 72000.00, '2018-06-20', 'Active'),
(106,3, 'Taylor', 'Finance', 58000.00, '2021-09-12', 'Active'),
(107, 5, 'Robert', 'Operations', 50000.00, '2023-02-18', 'Active'),
(108,2, 'Olivia', 'HR', 62000.00, '2019-04-30', 'Inactive'),
(109, 4, 'Daniel', 'Marketing', 57000.00, '2020-12-15', 'Active'),
(110,1, 'Sophia', 'IT', 68000.00, '2022-08-22', 'Active'),
(111,5, 'James', 'Operations', 53000.00, '2021-05-11', 'Inactive'),
(112,3, 'Emma', 'Finance', 61000.00, '2023-06-01', 'Active');
Create table department (
    DepartmentID int,
    Departmentname varchar(50)
);
insert into Employee values(1,'IT'),
(2,'HR'),
(3,'Finance'),
(4,'Marketing'),
(5,'Operations');

CREATE OR REPLACE VIEW EmployeeActive AS
SELECT
    e.EmpID,
    e.EmpName,
    d.DepartmentName,
    e.Salary,
    e.HireDate,
    e.Statu
FROM Employee AS e
JOIN Department AS d
    ON d.DepartmentID = e.DepartmentID
WHERE e.Statu = 'Active';
```

Task 11

Generate month-wise sales revenue.

```
Create table Sales (
    SaleID int primary Key,
    CustomerID int,
    ProductID int,
    SaleDate date,
    Quantity int,
    TotalAmount decimal(12,2)
);
insert into Sales(SaleID, CustomerID, ProductID, SaleDate, Quantity, TotalAmount)
values(1001, 201, 301, '2024-01-05', 2, 1200.00),
(1002, 202, 302, '2024-01-10', 1, 750.00),
(1003, 203, 303, '2024-01-15', 5, 2500.00),
(1004, 204, 304, '2024-02-01', 3, 1800.00),
(1005, 205, 305, '2024-02-08', 4, 3200.00),
(1006, 206, 306, '2024-02-15', 2, 1400.00),
(1007, 207, 307, '2024-03-01', 6, 4200.00),
(1008, 208, 308, '2024-03-10', 1, 950.00),
(1009, 209, 309, '2024-03-18', 7, 5600.00),
(1010, 210, 310, '2024-03-25', 3, 2100.00);

SELECT
    YEAR(SaleDate) AS Sales_Year,
    MONTH(SaleDate) AS Sales_Month,
    MONTHNAME(SaleDate) AS Month_Name,
    SUM(TotalAmount) AS Monthly_Revenue
FROM Sales
WHERE YEAR(SaleDate) = 2024
GROUP BY
    YEAR(SaleDate),
    MONTH(SaleDate),
    MONTHNAME(SaleDate)
ORDER BY
    Sales_Year,
    Sales_Month;
```

Task 12

Find products contributing to 80% of total revenue.

```
Create table Product (
    ProductID int primary key,
    ProductName varchar(150),
    CategoryID int,
    UnitPrice decimal(10,2)
);
insert into Product (ProductID, ProductName, CategoryID, UnitPrice) values
(301, 'Laptop', 1, 55000.00),
(302, 'Desktop Computer', 1, 48000.00),
(303, 'Wireless Mouse', 2, 850.00),
(304, 'Mechanical Keyboard', 2, 2200.00),
(305, 'LED Monitor 24 Inch', 3, 9500.00),
(306, 'USB Flash Drive 64GB', 4, 700.00),
(307, 'External Hard Drive 1TB', 4, 4200.00),
(308, 'Laser Printer', 5, 13500.00),
(309, 'Webcam HD', 6, 1800.00),
(310, 'Bluetooth Speaker', 7, 3200.00),
(311, 'Wireless Headphones', 7, 4500.00),
(312, 'Smartphone', 8, 28000.00);
Create table Sales (
    SaleID int primary Key,
    CustomerID int,
    ProductID int,
    SaleDate date,
    Quantity int,
    TotalAmount decimal(12,2)
);
insert into Sales(SaleID, CustomerID, ProductID, SaleDate, Quantity, TotalAmount)
values(1001, 201, 301, '2024-01-05', 2, 1200.00),
(1002, 202, 302, '2024-01-10', 1, 750.00),
(1003, 203, 303, '2024-01-15', 5, 2500.00),
(1004, 204, 304, '2024-02-01', 3, 1800.00),
(1005, 205, 305, '2024-02-08', 4, 3200.00),
(1006, 206, 306, '2024-02-15', 2, 1400.00),
(1007, 207, 307, '2024-03-01', 6, 4200.00),
(1008, 208, 308, '2024-03-10', 1, 950.00),
(1009, 209, 309, '2024-03-18', 7, 5600.00),
(1010, 210, 310, '2024-03-25', 3, 2100.00);

WITH ProductRevenue AS (
    SELECT
        p.ProductID,
        p.ProductName,
        SUM(s.TotalAmount) AS Revenue
    FROM Product AS p
    JOIN Sales AS s
        ON s.ProductID = p.ProductID
    GROUP BY
        p.ProductID,
        p.ProductName
),
RevenueShare AS (
    SELECT
        ProductID,
        ProductName,
        Revenue,
        SUM(Revenue) OVER (
            ORDER BY Revenue DESC
            ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
        ) AS Cumulative_Revenue,
        SUM(Revenue) OVER () AS Total_Revenue
    FROM ProductRevenue
)
SELECT
    ProductID,
    ProductName,
    Revenue,
    ROUND(
        100 * Cumulative_Revenue / NULLIF(Total_Revenue, 0),
        2
    ) AS Cumulative_Percent
FROM RevenueShare
WHERE Cumulative_Revenue <= 0.80 * Total_Revenue
    OR Cumulative_Revenue - Revenue < 0.80 * Total_Revenue
ORDER BY Revenue DESC;
```

Task 13

Display products that have never been sold.

```
Create table Product (
    ProductID int primary key,
    ProductName varchar(150),
    CategoryID int,
    UnitPrice decimal(10,2)
);
insert into Product (ProductID, ProductName, CategoryID, UnitPrice) values
(301, 'Laptop', 1, 55000.00),
(302, 'Desktop Computer', 1, 48000.00),
(303, 'Wireless Mouse', 2, 850.00),
(304, 'Mechanical Keyboard', 2, 2200.00),
(305, 'LED Monitor 24 Inch', 3, 9500.00),
(306, 'USB Flash Drive 64GB', 4, 700.00),
(307, 'External Hard Drive 1TB', 4, 4200.00),
(308, 'Laser Printer', 5, 13500.00),
(309, 'Webcam HD', 6, 1800.00),
(310, 'Bluetooth Speaker', 7, 3200.00),
(311, 'Wireless Headphones', 7, 4500.00),
(312, 'Smartphone', 8, 28000.00);
Create table Sales (
    SaleID int primary Key,
    CustomerID int,
    ProductID int,
    SaleDate date,
    Quantity int,
    TotalAmount decimal(12,2)
);
insert into Sales(SaleID, CustomerID, ProductID, SaleDate, Quantity, TotalAmount)
values(1001, 201, 301, '2024-01-05', 2, 1200.00),
(1002, 202, 302, '2024-01-10', 1, 750.00),
(1003, 203, 303, '2024-01-15', 5, 2500.00),
(1004, 204, 304, '2024-02-01', 3, 1800.00),
(1005, 205, 305, '2024-02-08', 4, 3200.00),
(1006, 206, 306, '2024-02-15', 2, 1400.00),
(1007, 207, 307, '2024-03-01', 6, 4200.00),
(1008, 208, 308, '2024-03-10', 1, 950.00),
(1009, 209, 309, '2024-03-18', 7, 5600.00),
(1010, 210, 310, '2024-03-25', 3, 2100.00);
Create table Category (
    CategoryID int primary key,
    CategoryName varchar(100)
);
insert into Category (CategoryID, CategoryName) values
(1, 'Computers'),
(2, 'Computer Accessories'),
(3, 'Monitors'),
(4, 'Storage Devices'),
(5, 'Printers'),
(6, 'Cameras'),
(7, 'Audio Devices'),
(8, 'Mobile Phones'),
(9, 'Networking'),
(10, 'Gaming'),
(11, 'Office Equipment'),
(12, 'Smart Home');

SELECT
    p.ProductID,
    p.ProductName,
    c.CategoryName
FROM Product AS p
JOIN Category AS c
    ON c.CategoryID = p.CategoryID
WHERE NOT EXISTS (
    SELECT 1
    FROM Sales AS s
    WHERE s.ProductID = p.ProductID
)
ORDER BY p.ProductID;
```

Task 14

Identify customers purchasing from multiple categories.

```
create table Customer (CustomerID int primary key, CustomerName varchar(100), Gender varchar(20), City varchar(100),
Region varchar(50), JoinDate date);
insert into Customer (CustomerID, CustomerName, Gender, City, Region, JoinDate) values
(201, 'Smith', 'Male', 'New York', 'North', '2022-01-15'),
(202, 'Johnson', 'Female', 'Los Angeles', 'West', '2022-03-20'),
(203, 'Michael', 'Male', 'Chicago', 'North', '2022-05-10'),
(204, 'Williams', 'Female', 'Houston', 'South', '2022-07-18'),
(205, 'David', 'Male', 'Phoenix', 'West', '2022-09-12'),
(206, 'Taylor', 'Female', 'Philadelphia', 'East', '2022-10-25'),
(207, 'Anderson', 'Male', 'San Antonio', 'South', '2023-01-08'),
(208, 'Thomas', 'Female', 'San Diego', 'West', '2023-02-14'),
(209, 'Martinez', 'Male', 'Dallas', 'South', '2023-04-30'),
(210, 'Davis', 'Female', 'San Jose', 'West', '2023-06-11'),
(211, 'Wilson', 'Male', 'Austin', 'South', '2023-08-19'),
(212, 'Emma', 'Female', 'Jacksonville', 'East', '2023-11-05');
Create table Category (
    CategoryID int primary key,
    CategoryName varchar(100)
);
insert into Category (CategoryID, CategoryName) values
(1, 'Computers'),
(2, 'Computer Accessories'),
(3, 'Monitors'),
(4, 'Storage Devices'),
(5, 'Printers'),
(6, 'Cameras'),
(7, 'Audio Devices'),
(8, 'Mobile Phones'),
(9, 'Networking'),
(10, 'Gaming'),
(11, 'Office Equipment'),
(12, 'Smart Home');
Create table Product (
    ProductID int primary key,
    ProductName varchar(150),
    CategoryID int,
    UnitPrice decimal(10,2)
);
insert into Product (ProductID, ProductName, CategoryID, UnitPrice) values
(301, 'Laptop', 1, 55000.00),
(302, 'Desktop Computer', 1, 48000.00),
(303, 'Wireless Mouse', 2, 850.00),
(304, 'Mechanical Keyboard', 2, 2200.00),
(305, 'LED Monitor 24 Inch', 3, 9500.00),
(306, 'USB Flash Drive 64GB', 4, 700.00),
(307, 'External Hard Drive 1TB', 4, 4200.00),
(308, 'Laser Printer', 5, 13500.00),
(309, 'Webcam HD', 6, 1800.00),
(310, 'Bluetooth Speaker', 7, 3200.00),
(311, 'Wireless Headphones', 7, 4500.00),
(312, 'Smartphone', 8, 28000.00);
Create table Sales (
    SaleID int primary Key,
    CustomerID int,
    ProductID int,
    SaleDate date,
    Quantity int,
    TotalAmount decimal(12,2)
);
insert into Sales(SaleID, CustomerID, ProductID, SaleDate, Quantity, TotalAmount)
values(1001, 201, 301, '2024-01-05', 2, 1200.00),
(1002, 202, 302, '2024-01-10', 1, 750.00),
(1003, 203, 303, '2024-01-15', 5, 2500.00),
(1004, 204, 304, '2024-02-01', 3, 1800.00),
(1005, 205, 305, '2024-02-08', 4, 3200.00),
(1006, 206, 306, '2024-02-15', 2, 1400.00),
(1007, 207, 307, '2024-03-01', 6, 4200.00),
(1008, 208, 308, '2024-03-10', 1, 950.00),
(1009, 209, 309, '2024-03-18', 7, 5600.00),
(1010, 210, 310, '2024-03-25', 3, 2100.00);

SELECT
    c.CustomerID,
    c.CustomerName,
    COUNT(DISTINCT p.CategoryID) AS Categories_Purchased
FROM Customer AS c
JOIN Sales AS s
    ON s.CustomerID = c.CustomerID
JOIN Product AS p
```

```
        ON p.ProductID = s.ProductID
GROUP BY
    c.CustomerID,
    c.CustomerName
HAVING COUNT(DISTINCT p.CategoryID) > 1
ORDER BY Categories_Purchased DESC, c.CustomerName;
```

Task 15

Find the second highest selling product in every category.

```
Create table Category (
    CategoryID int primary key,
    CategoryName varchar(100)
);
insert into Category (CategoryID, CategoryName) values
(1, 'Computers'),
(2, 'Computer Accessories'),
(3, 'Monitors'),
(4, 'Storage Devices'),
(5, 'Printers'),
(6, 'Cameras'),
(7, 'Audio Devices'),
(8, 'Mobile Phones'),
(9, 'Networking'),
(10, 'Gaming'),
(11, 'Office Equipment'),
(12, 'Smart Home');
Create table Product (
    ProductID int primary key,
    ProductName varchar(150),
    CategoryID int,
    UnitPrice decimal(10,2)
);
insert into Product (ProductID, ProductName, CategoryID, UnitPrice) values
(301, 'Laptop', 1, 55000.00),
(302, 'Desktop Computer', 1, 48000.00),
(303, 'Wireless Mouse', 2, 850.00),
(304, 'Mechanical Keyboard', 2, 2200.00),
(305, 'LED Monitor 24 Inch', 3, 9500.00),
(306, 'USB Flash Drive 64GB', 4, 700.00),
(307, 'External Hard Drive 1TB', 4, 4200.00),
(308, 'Laser Printer', 5, 13500.00),
(309, 'Webcam HD', 6, 1800.00),
(310, 'Bluetooth Speaker', 7, 3200.00),
(311, 'Wireless Headphones', 7, 4500.00),
(312, 'Smartphone', 8, 28000.00);
WITH ProductSales AS (
    SELECT
        c.CategoryID,
        c.CategoryName,
        p.ProductID,
        p.ProductName,
        SUM(s.TotalAmount) AS Total_Sales
    FROM Category AS c
    JOIN Product AS p
        ON p.CategoryID = c.CategoryID
    JOIN Sales AS s
        ON s.ProductID = p.ProductID
    GROUP BY
        c.CategoryID,
        c.CategoryName,
        p.ProductID,
        p.ProductName
),
RankedSales AS (
    SELECT
        ProductSales.*,
        DENSE_RANK() OVER (
            PARTITION BY CategoryID
            ORDER BY Total_Sales DESC
        ) AS Sales_Rank
    FROM ProductSales
)
SELECT
    CategoryName,
    ProductName,
    Total_Sales
FROM RankedSales
WHERE Sales_Rank = 2
ORDER BY CategoryName;
```


Task 16

Generate customer lifetime value.

```
create table Customer (CustomerID int primary key, CustomerName varchar(100), Gender varchar(20), City varchar(100),
Region varchar(50), JoinDate date);
insert into Customer (CustomerID, CustomerName, Gender, City, Region, JoinDate) values
(201, 'Smith', 'Male', 'New York', 'North', '2022-01-15'),
(202, 'Johnson', 'Female', 'Los Angeles', 'West', '2022-03-20'),
(203, 'Michael', 'Male', 'Chicago', 'North', '2022-05-10'),
(204, 'Williams', 'Female', 'Houston', 'South', '2022-07-18'),
(205, 'David', 'Male', 'Phoenix', 'West', '2022-09-12'),
(206, 'Taylor', 'Female', 'Philadelphia', 'East', '2022-10-25'),
(207, 'Anderson', 'Male', 'San Antonio', 'South', '2023-01-08'),
(208, 'Thomas', 'Female', 'San Diego', 'West', '2023-02-14'),
(209, 'Martinez', 'Male', 'Dallas', 'South', '2023-04-30'),
(210, 'Davis', 'Female', 'San Jose', 'West', '2023-06-11'),
(211, 'Wilson', 'Male', 'Austin', 'South', '2023-08-19'),
(212, 'Emma', 'Female', 'Jacksonville', 'East', '2023-11-05');
Create table Sales (
    SaleID int primary Key,
    CustomerID int,
    ProductID int,
    SaleDate date,
    Quantity int,
    TotalAmount decimal(12,2)
);
insert into Sales(SaleID, CustomerID, ProductID, SaleDate, Quantity, TotalAmount)
values(1001, 201, 301, '2024-01-05', 2, 1200.00),
(1002, 202, 302, '2024-01-10', 1, 750.00),
(1003, 203, 303, '2024-01-15', 5, 2500.00),
(1004, 204, 304, '2024-02-01', 3, 1800.00),
(1005, 205, 305, '2024-02-08', 4, 3200.00),
(1006, 206, 306, '2024-02-15', 2, 1400.00),
(1007, 207, 307, '2024-03-01', 6, 4200.00),
(1008, 208, 308, '2024-03-10', 1, 950.00),
(1009, 209, 309, '2024-03-18', 7, 5600.00),
(1010, 210, 310, '2024-03-25', 3, 2100.00);

SELECT
    c.CustomerID,
    c.CustomerName,
    COUNT(DISTINCT s.SaleID) AS Total_Orders,
    COALESCE(SUM(s.TotalAmount), 0) AS Customer_Lifetime_Value
FROM Customer AS c
LEFT JOIN Sales AS s
    ON s.CustomerID = c.CustomerID
GROUP BY
    c.CustomerID,
    c.CustomerName
ORDER BY Customer_Lifetime_Value DESC;
```

Task 17

Display daily sales with previous day sales and next day sales.

```
Create table Sales (
  SaleID int primary Key,
  CustomerID int,
  ProductID int,
  SaleDate date,
  Quantity int,
  TotalAmount decimal(12,2)
);
insert into Sales(SaleID, CustomerID, ProductID, SaleDate, Quantity, TotalAmount)
values(1001, 201, 301, '2024-01-05', 2, 1200.00),
(1002, 202, 302, '2024-01-10', 1, 750.00),
(1003, 203, 303, '2024-01-15', 5, 2500.00),
(1004, 204, 304, '2024-02-01', 3, 1800.00),
(1005, 205, 305, '2024-02-08', 4, 3200.00),
(1006, 206, 306, '2024-02-15', 2, 1400.00),
(1007, 207, 307, '2024-03-01', 6, 4200.00),
(1008, 208, 308, '2024-03-10', 1, 950.00),
(1009, 209, 309, '2024-03-18', 7, 5600.00),
(1010, 210, 310, '2024-03-25', 3, 2100.00);

WITH DailySales AS (
  SELECT
    SaleDate,
    SUM(TotalAmount) AS Daily_Revenue
  FROM Sales
  GROUP BY SaleDate
)
SELECT
  SaleDate,
  Daily_Revenue,
  LAG(Daily_Revenue) OVER (
    ORDER BY SaleDate
  ) AS Previous_Day_Revenue,
  LEAD(Daily_Revenue) OVER (
    ORDER BY SaleDate
  ) AS Next_Day_Revenue
FROM DailySales
ORDER BY SaleDate;
```

Task 18

Find the top five customers in each region.

```
create table Customer (CustomerID int primary key, CustomerName varchar(100), Gender varchar(20), City varchar(100),
Region varchar(50), JoinDate date);
insert into Customer (CustomerID, CustomerName, Gender, City, Region, JoinDate) values
(201, 'Smith', 'Male', 'New York', 'North', '2022-01-15'),
(202, 'Johnson', 'Female', 'Los Angeles', 'West', '2022-03-20'),
(203, 'Michael', 'Male', 'Chicago', 'North', '2022-05-10'),
(204, 'Williams', 'Female', 'Houston', 'South', '2022-07-18'),
(205, 'David', 'Male', 'Phoenix', 'West', '2022-09-12'),
(206, 'Taylor', 'Female', 'Philadelphia', 'East', '2022-10-25'),
(207, 'Anderson', 'Male', 'San Antonio', 'South', '2023-01-08'),
(208, 'Thomas', 'Female', 'San Diego', 'West', '2023-02-14'),
(209, 'Martinez', 'Male', 'Dallas', 'South', '2023-04-30'),
(210, 'Davis', 'Female', 'San Jose', 'West', '2023-06-11'),
(211, 'Wilson', 'Male', 'Austin', 'South', '2023-08-19'),
(212, 'Emma', 'Female', 'Jacksonville', 'East', '2023-11-05');
Create table Sales (
    SaleID int primary Key,
    CustomerID int,
    ProductID int,
    SaleDate date,
    Quantity int,
    TotalAmount decimal(12,2)
);
insert into Sales(SaleID, CustomerID, ProductID, SaleDate, Quantity, TotalAmount)
values(1001, 201, 301, '2024-01-05', 2, 1200.00),
(1002, 202, 302, '2024-01-10', 1, 750.00),
(1003, 203, 303, '2024-01-15', 5, 2500.00),
(1004, 204, 304, '2024-02-01', 3, 1800.00),
(1005, 205, 305, '2024-02-08', 4, 3200.00),
(1006, 206, 306, '2024-02-15', 2, 1400.00),
(1007, 207, 307, '2024-03-01', 6, 4200.00),
(1008, 208, 308, '2024-03-10', 1, 950.00),
(1009, 209, 309, '2024-03-18', 7, 5600.00),
(1010, 210, 310, '2024-03-25', 3, 2100.00);

WITH CustomerRevenue AS (
    SELECT
        c.Region,
        c.CustomerID,
        c.CustomerName,
        SUM(s.TotalAmount) AS Total_Revenue
    FROM Customer AS c
    JOIN Sales AS s
        ON s.CustomerID = c.CustomerID
    GROUP BY
        c.Region,
        c.CustomerID,
        c.CustomerName
),
RankedCustomers AS (
    SELECT
        CustomerRevenue.*,
        ROW_NUMBER() OVER (
            PARTITION BY Region
            ORDER BY Total_Revenue DESC, CustomerID
        ) AS Region_Rank
    FROM CustomerRevenue
)
SELECT
    Region,
    CustomerName,
    Total_Revenue,
    Region_Rank
FROM RankedCustomers
WHERE Region_Rank <= 5
ORDER BY Region, Region_Rank;
```

Task 19

```
Create a View for monthly sales reporting. */
Create table Sales (
    SaleID int primary Key,
    CustomerID int,
    ProductID int,
    SaleDate date,
    Quantity int,
    TotalAmount decimal(12,2)
);
insert into Sales(SaleID, CustomerID, ProductID, SaleDate, Quantity, TotalAmount)
values(1001, 201, 301, '2024-01-05', 2, 1200.00),
(1002, 202, 302, '2024-01-10', 1, 750.00),
(1003, 203, 303, '2024-01-15', 5, 2500.00),
(1004, 204, 304, '2024-02-01', 3, 1800.00),
(1005, 205, 305, '2024-02-08', 4, 3200.00),
(1006, 206, 306, '2024-02-15', 2, 1400.00),
(1007, 207, 307, '2024-03-01', 6, 4200.00),
(1008, 208, 308, '2024-03-10', 1, 950.00),
(1009, 209, 309, '2024-03-18', 7, 5600.00),
(1010, 210, 310, '2024-03-25', 3, 2100.00);

CREATE OR REPLACE VIEW MonthlySales AS
SELECT
    DATE_FORMAT(SaleDate, '%Y-%m') AS Sales_Month,
    COUNT(*) AS Total_Orders,
    SUM(TotalAmount) AS Total_Revenue,
    SUM(Quantity) AS Total_Units_Sold
FROM Sales
GROUP BY DATE_FORMAT(SaleDate, '%Y-%m');
```

Task 20

Generate a CTE to simplify regional sales analysis.

```
create table Customer (CustomerID int primary key, CustomerName varchar(100), Gender varchar(20), City varchar(100),
Region varchar(50), JoinDate date);
insert into Customer (CustomerID, CustomerName, Gender, City, Region, JoinDate) values
(201, 'Smith', 'Male', 'New York', 'North', '2022-01-15'),
(202, 'Johnson', 'Female', 'Los Angeles', 'West', '2022-03-20'),
(203, 'Michael', 'Male', 'Chicago', 'North', '2022-05-10'),
(204, 'Williams', 'Female', 'Houston', 'South', '2022-07-18'),
(205, 'David', 'Male', 'Phoenix', 'West', '2022-09-12'),
(206, 'Taylor', 'Female', 'Philadelphia', 'East', '2022-10-25'),
(207, 'Anderson', 'Male', 'San Antonio', 'South', '2023-01-08'),
(208, 'Thomas', 'Female', 'San Diego', 'West', '2023-02-14'),
(209, 'Martinez', 'Male', 'Dallas', 'South', '2023-04-30'),
(210, 'Davis', 'Female', 'San Jose', 'West', '2023-06-11'),
(211, 'Wilson', 'Male', 'Austin', 'South', '2023-08-19'),
(212, 'Emma', 'Female', 'Jacksonville', 'East', '2023-11-05');
Create table Sales (
    SaleID int primary Key,
    CustomerID int,
    ProductID int,
    SaleDate date,
    Quantity int,
    TotalAmount decimal(12,2)
);
insert into Sales(SaleID, CustomerID, ProductID, SaleDate, Quantity, TotalAmount)
values(1001, 201, 301, '2024-01-05', 2, 1200.00),
(1002, 202, 302, '2024-01-10', 1, 750.00),
(1003, 203, 303, '2024-01-15', 5, 2500.00),
(1004, 204, 304, '2024-02-01', 3, 1800.00),
(1005, 205, 305, '2024-02-08', 4, 3200.00),
(1006, 206, 306, '2024-02-15', 2, 1400.00),
(1007, 207, 307, '2024-03-01', 6, 4200.00),
(1008, 208, 308, '2024-03-10', 1, 950.00),
(1009, 209, 309, '2024-03-18', 7, 5600.00),
(1010, 210, 310, '2024-03-25', 3, 2100.00);

WITH RegionalSales AS (
    SELECT
        c.Region,
        COUNT(DISTINCT c.CustomerID) AS Total_Customers,
        COUNT(s.SaleID) AS Total_Orders,
        SUM(s.TotalAmount) AS Total_Revenue
    FROM Customer AS c
    JOIN Sales AS s
        ON s.CustomerID = c.CustomerID
    GROUP BY c.Region
)
SELECT
    Region,
    Total_Customers,
    Total_Orders,
    Total_Revenue,
    ROUND(
        Total_Revenue / NULLIF(Total_Customers, 0),
        2
    ) AS Revenue_Per_Customer
FROM RegionalSales
ORDER BY Total_Revenue DESC;
```

Task 21

Compare today's batch record count with yesterday's batch.

```
Create table Batch_Master (
    Batch_ID int primary key,
    Batch_Name varchar(100),
    Batch_Date date,
    Load_Status varchar(20)
);
insert into Batch_Master (Batch_ID, Batch_Name, Batch_Date, Load_Status)
values(101, 'Customer Data Load', '2026-08-01', 'Success'),
(102, 'Sales Data Load', '2026-08-02', 'Failed'),
(103, 'Product Data Load', '2026-08-03', 'Success'),
(104, 'Inventory Data Load', '2026-08-04', 'Partial'),
(105, 'Finance Data Load', '2026-08-05', 'Success');
create table Staging_Customer (Customer_ID int, Customer_Name varchar(100), Email varchar(100), Phone varchar(20), City varchar(100),
Validation_Status varchar(20));
insert into Staging_Customer (Customer_ID, Customer_Name, Email, Phone, City, Batch_ID, Validation_Status)
values(1001, 'Smith', 'john.smith@email.com', '9876543210', 'New York', 101, 'Valid'),
(1002, 'Emily Johnson', 'emily.johnson@email.com', '9876543211', 'Los Angeles', 101, 'Valid'),
(1003, 'Michael ', 'michael.brown@email.com', '9876543212', 'Chicago', 102, 'Invalid'),
(1004, 'Davis', 'sophia.davis@email.com', '9876543213', 'Houston', 102, 'Valid'),
(1005, 'William', 'william.wilson@email.com', '9876543214', 'Phoenix', 103, 'Valid'),
(1006, 'Olivia', 'olivia.taylor@email.com', '9876543215', 'Philadelphia', 103, 'Pending'),
(1007, 'Anderson', 'james.anderson@email.com', '9876543216', 'San Antonio', 104, 'Valid'),
(1008, 'Charlotte', 'charlotte.thomas@email.com', '9876543217', 'San Diego', 104, 'Invalid'),
(1009, 'Benjamin', 'benjamin.moore@email.com', '9876543218', 'Dallas', 105, 'Valid'),
(1010, 'Amelia', 'amelia.martin@email.com', '9876543219', 'San Jose', 105, 'Pending'),
(1011, 'Henry', 'henry.jackson@email.com', '9876543220', 'Austin', 101, 'Valid'),
(1012, 'Evelyn', 'evelyn.white@email.com', '9876543221', 'Jacksonville', 102, 'Invalid');

WITH BatchCounts AS (
    SELECT
        b.Batch_ID,
        b.Batch_Date,
        COUNT(s.Customer_ID) AS Record_Count
    FROM Batch_Master AS b
    LEFT JOIN Staging_Customer AS s
        ON s.Batch_ID = b.Batch_ID
    GROUP BY
        b.Batch_ID,
        b.Batch_Date
)
SELECT
    Batch_Date,
    Record_Count,
    LAG(Record_Count) OVER (
        ORDER BY Batch_Date
    ) AS Previous_Day_Count,
    Record_Count - LAG(Record_Count) OVER (
        ORDER BY Batch_Date
    ) AS Count_Change
FROM BatchCounts
ORDER BY Batch_Date;
```

Task 22

Identify duplicate records loaded into the staging table.

```
create table Staging_Customer (Customer_ID int, Customer_Name varchar(100), Email varchar(100), Phone varchar(20), City varchar(100), Validation_Status varchar(20));
insert into Staging_Customer (Customer_ID, Customer_Name, Email, Phone, City, Batch_ID, Validation_Status)
values(1001, 'Smith', 'smith@email.com', '9876543210', 'New York', 101, 'Valid'),
(1002, 'Emily', 'emily@email.com', '9876543211', 'Los Angeles', 101, 'Valid'),
(1003, 'Michael ', 'michael@email.com', '9876543212', 'Chicago', 102, 'Invalid'),
(1004, 'Davis', 'davis@email.com', '9876543213', 'Houston', 102, 'Valid'),
(1005, 'William', 'william@email.com', '9876543214', 'Phoenix', 103, 'Valid'),
(1006, 'Olivia', 'olivia@email.com', '9876543215', 'Philadelphia', 103, 'Pending'),
(1007, ' Anderson', 'anderson@email.com', '9876543216', 'San Antonio', 104, 'Valid'),
(1008, 'Charlotte', 'charlotte@email.com', '9876543217', 'San Diego', 104, 'Invalid'),
(1009, 'Benjamin', 'benjamin@email.com', '9876543218', 'Dallas', 105, 'Valid'),
(1010, 'Amelia', 'amelia@email.com', '9876543219', 'San Jose', 105, 'Pending'),
(1011, 'Henry', 'henry@email.com', '9876543220', 'Austin', 101, 'Valid'),
(1012, 'Evelyn', 'evelyn@email.com', '9876543221', 'Jacksonville', 102, 'Invalid');

SELECT
    Customer_ID,
    COUNT(*) AS Occurrences
FROM Staging_Customer
GROUP BY Customer_ID
HAVING COUNT(*) > 1
ORDER BY Customer_ID;
```

Task 23

Find NULL values in mandatory columns.

```
create table Staging_Customer (Customer_ID int, Customer_Name varchar(100), Email varchar(100), Phone varchar(20), City varchar(100), Validation_Status varchar(20));
insert into Staging_Customer (Customer_ID, Customer_Name, Email, Phone, City, Batch_ID, Validation_Status)
values (1001, 'Smith', 'smith@email.com', '9876543210', 'New York', 101, 'Valid'),
(1002, 'Emily', 'emily@email.com', '9876543211', 'Los Angeles', 101, null),
(1003, 'Michael ', 'michael@email.com', '9876543212', 'Chicago', 102, 'Invalid'),
(1004, 'Davis', 'davis@email.com', '9876543213', 'Houston', 102, 'Valid'),
(1005, 'William', 'william@email.com', '9876543214', 'Phoenix', 103, 'Valid'),
(1006, 'Olivia', 'olivia@email.com', '9876543215', 'Philadelphia', null, 'Pending'),
(1007, 'Anderson', 'anderson@email.com', '9876543216', 'San Antonio', 104, 'Valid'),
(1008, 'Charlotte', 'charlotte@email.com', '9876543217', 'San Diego', 104, 'Invalid'),
(1009, 'Benjamin', 'benjamin@email.com', '9876543218', 'Dallas', 105, 'Valid'),
(1010, 'Amelia', 'amelia@email.com', 'null', 'San Jose', 105, 'Pending'),
(1011, 'Henry', 'henry@email.com', '9876543220', 'Austin', 101, 'Valid'),
(1012, 'Evelyn', null, '9876543221', 'Jacksonville', 102, 'Invalid');

SELECT
    Customer_ID,
    Customer_Name,
    Email,
    Phone,
    Batch_ID
FROM Staging_Customer
WHERE Customer_Name IS NULL
    OR Email IS NULL
    OR Phone IS NULL
ORDER BY Customer_ID;
```


Task 24

Generate batch-wise load statistics.

```
Create table Batch_Master (
    Batch_ID int primary key,
    Batch_Name varchar(100),
    Batch_Date date,
    Load_Status varchar(20)
);
insert into Batch_Master (Batch_ID, Batch_Name, Batch_Date, Load_Status)
values(101, 'Customer Data Load', '2026-08-01', 'Success'),
(102, 'Sales Data Load', '2026-08-02', 'Failed'),
(103, 'Product Data Load', '2026-08-03', 'Success'),
(104, 'Inventory Data Load', '2026-08-04', 'Partial'),
(105, 'Finance Data Load', '2026-08-05', 'Success');
create table Staging_Customer (Customer_ID int, Customer_Name varchar(100), Email varchar(100), Phone varchar(20), City varchar(100),
Validation_Status varchar(20));
insert into Staging_Customer (Customer_ID, Customer_Name, Email, Phone, City, Batch_ID, Validation_Status)
values(1001, 'Smith', 'john.smith@email.com', '9876543210', 'New York', 101, 'Valid'),
(1002, 'Emily Johnson', 'emily.johnson@email.com', '9876543211', 'Los Angeles', 101, 'Valid'),
(1003, 'Michael ', 'michael.brown@email.com', '9876543212', 'Chicago', 102, 'Invalid'),
(1004, 'Davis', 'sophia.davis@email.com', '9876543213', 'Houston', 102, 'Valid'),
(1005, 'William', 'william.wilson@email.com', '9876543214', 'Phoenix', 103, 'Valid'),
(1006, 'Olivia', 'olivia.taylor@email.com', '9876543215', 'Philadelphia', 103, 'Pending'),
(1007, 'Anderson', 'james.anderson@email.com', '9876543216', 'San Antonio', 104, 'Valid'),
(1008, 'Charlotte', 'charlotte.thomas@email.com', '9876543217', 'San Diego', 104, 'Invalid'),
(1009, 'Benjamin', 'benjamin.moore@email.com', '9876543218', 'Dallas', 105, 'Valid'),
(1010, 'Amelia', 'amelia.martin@email.com', '9876543219', 'San Jose', 105, 'Pending'),
(1011, 'Henry', 'henry.jackson@email.com', '9876543220', 'Austin', 101, 'Valid'),
(1012, 'Evelyn', 'evelyn.white@email.com', '9876543221', 'Jacksonville', 102, 'Invalid');
Create table Rejected_Records (
    Reject_ID int primary key,
    Customer_ID int,
    Batch_ID int,
    Reject_Reason varchar(200)
);
insert into Rejected_Records (Reject_ID, Customer_ID, Batch_ID, Reject_Reason)
values(1, 1003, 102, 'Duplicate Customer ID'),
(2, 1008, 104, 'Invalid Email Address'),
(3, 1013, 106, 'Customer Name is NULL'),
(4, 1014, 106, 'Email is NULL'),
(5, 1015, 107, 'Phone Number is NULL'),
(6, 1016, 107, 'Multiple Mandatory Fields Missing'),
(7, 1017, 108, 'Phone Number is NULL'),
(8, 1006, 103, 'Invalid Customer Data');

WITH StagingStats AS (
    SELECT
        Batch_ID,
        COUNT(*) AS Total_Records,
        SUM(Validation_Status = 'Valid') AS Successful_Records
    FROM Staging_Customer
    GROUP BY Batch_ID
),
RejectedStats AS (
    SELECT
        Batch_ID,
        COUNT(*) AS Rejected_Records
    FROM Rejected_Records
    GROUP BY Batch_ID
)
SELECT
    b.Batch_ID,
    b.Batch_Date,
    COALESCE(s.Total_Records, 0) AS Total_Records,
    COALESCE(s.Successful_Records, 0) AS Successful_Records,
    COALESCE(r.Rejected_Records, 0) AS Rejected_Records
FROM Batch_Master AS b
LEFT JOIN StagingStats AS s
    ON s.Batch_ID = b.Batch_ID
LEFT JOIN RejectedStats AS r
    ON r.Batch_ID = b.Batch_ID
ORDER BY b.Batch_Date;
```

Task 25

Find records rejected during ETL processing.

```
Create table Rejected_Records (  
    Reject_ID int primary key,  
    Customer_ID int,  
    Batch_ID int,  
    Reject_Reason varchar(200)  
);  
insert into Rejected_Records (Reject_ID, Customer_ID, Batch_ID, Reject_Reason)  
values(1, 1003, 102, 'Duplicate Customer ID'),  
(2, 1008, 104, 'Invalid Email Address'),  
(3, 1013, 106, 'Customer Name is NULL'),  
(4, 1014, 106, 'Email is NULL'),  
(5, 1015, 107, 'Phone Number is NULL'),  
(6, 1016, 107, 'Multiple Mandatory Fields Missing'),  
(7, 1017, 108, 'Phone Number is NULL'),  
(8, 1006, 103, 'Invalid Customer Data');  
  
SELECT  
    Customer_ID,  
    Batch_ID,  
    Reject_Reason  
FROM Rejected_Records  
WHERE Reject_Reason IS NOT NULL  
ORDER BY Batch_ID, Customer_ID;
```

Task 26

```
Create a Temporary Table containing invalid records for further validation. */
create table Staging_Customer (Customer_ID int, Customer_Name varchar(100), Email varchar(100), Phone varchar(20), City varchar(100),
Validation_Status varchar(20));
insert into Staging_Customer (Customer_ID, Customer_Name, Email, Phone, City, Batch_ID, Validation_Status)
values(1001, 'Smith', 'smith@email.com', '9876543210', 'New York', 101, 'Valid'),
(1002, 'Emily', 'emily@email.com', '9876543211', 'Los Angeles', 101, 'Valid'),
(1003, 'Michael ', 'michael@email.com', '9876543212', 'Chicago', 102, 'Invalid'),
(1004, 'Davis', 'davis@email.com', '9876543213', 'Houston', 102, 'Valid'),
(1005, 'William', 'william@email.com', '9876543214', 'Phoenix', 103, 'Valid'),
(1006, 'Olivia', 'olivia@email.com', '9876543215', 'Philadelphia', 103, 'Pending'),
(1007, ' Anderson', 'anderson@email.com', '9876543216', 'San Antonio', 104, 'Valid'),
(1008, 'Charlotte', 'charlotte@email.com', '9876543217', 'San Diego', 104, 'Invalid'),
(1009, 'Benjamin', 'benjamin@email.com', '9876543218', 'Dallas', 105, 'Valid'),
(1010, 'Amelia', 'amelia@email.com', '9876543219', 'San Jose', 105, 'Pending'),
(1011, 'Henry', 'henry@email.com', '9876543220', 'Austin', 101, 'Valid'),
(1012, 'Evelyn', 'evelyn@email.com', '9876543221', 'Jacksonville', 102, 'Invalid');

CREATE TEMPORARY TABLE Temp_Invalid_Customers AS
SELECT
    Customer_ID,
    Customer_Name,
    Email,
    Phone,
    City,
    Batch_ID,
    Validation_Status
FROM Staging_Customer
WHERE Validation_Status = 'Invalid';
```

Task 27

Find source records that failed to reach the target table.

```
create table Source_Customer (Customer_ID int, Customer_Name varchar(100), Email varchar(100), Phone varchar(20), City varchar(100), Batch_ID int);
insert into Source_Customer (Customer_ID, Customer_Name, Email, Phone, City, Batch_ID)
values (2001, 'Robert', 'robert@email.com', '5551000001', 'New York', 101),
(2002, 'Jennifer', 'jennifer@email.com', '5551000002', 'Los Angeles', 101),
(2003, 'David', 'david@email.com', '5551000003', 'Chicago', 101),
(2004, 'Linda', 'linda@email.com', '5551000004', 'Houston', 101),
(2005, 'Richard', 'richard@email.com', '5551000005', 'Phoenix', 101);
create table Warehouse_Customer (Customer_ID int, Customer_Name varchar(100), Email varchar(100), Phone varchar(20), City varchar(100), Batch_ID int, Load_Date date);
insert into Warehouse_Customer (Customer_ID, Customer_Name, Email, Phone, City, Batch_ID, Load_Date)
values (2021, 'Mark', 'mark@email.com', '5551000021', 'Boston', 105, '2026-08-05'),
(2022, 'Betty', 'betty@email.com', '5551000022', 'Nashville', 105, '2026-08-05'),
(2023, 'Steven', 'steven@email.com', '5551000023', 'Detroit', 105, '2026-08-05'),
(2024, 'Helen', 'helen@email.com', '5551000024', 'Portland', 105, '2026-08-05'),
(2025, 'Paul', 'paul@email.com', '5551000025', 'Las Vegas', 105, '2026-08-05');

SELECT
    s.Customer_ID,
    s.Customer_Name
FROM Source_Customer AS s
WHERE NOT EXISTS (
    SELECT 1
    FROM Warehouse_Customer AS w
    WHERE w.Customer_ID = s.Customer_ID
)
ORDER BY s.Customer_ID;
```

Task 28

Identify records loaded multiple times into the warehouse.

```
create table Warehouse_Customer (Customer_ID int, Customer_Name varchar(100), Email varchar(100), Phone varchar(20), City varchar(100), Batch_ID int, Load_Date date);
insert into Warehouse_Customer (Customer_ID, Customer_Name, Email, Phone, City, Batch_ID, Load_Date)
values (2021, 'Mark', 'mark@email.com', '5551000021', 'Boston', 105, '2026-08-05'),
(2022, 'Betty', 'betty@email.com', '5551000022', 'Nashville', 105, '2026-08-05'),
(2023, 'Steven', 'steven@email.com', '5551000023', 'Detroit', 105, '2026-08-05'),
(2024, 'Helen', 'helen@email.com', '5551000024', 'Portland', 105, '2026-08-05'),
(2025, 'Paul', 'paul@email.com', '5551000025', 'Las Vegas', 105, '2026-08-05');

SELECT
    Customer_ID,
    COUNT(*) AS Load_Count
FROM Warehouse_Customer
GROUP BY Customer_ID
HAVING COUNT(*) > 1
ORDER BY Customer_ID;
```

Task 29

Generate source-to-target reconciliation report.

```
create table Source_Customer (Customer_ID int, Customer_Name varchar(100), Email varchar(100), Phone varchar(20), City varchar(100), Batch_ID int);
insert into Source_Customer (Customer_ID, Customer_Name, Email, Phone, City, Batch_ID)
values (2001, 'Robert', 'robert@email.com', '5551000001', 'New York', 101),
(2002, 'Jennifer', 'jennifer@email.com', '5551000002', 'Los Angeles', 101),
(2003, 'David', 'david@email.com', '5551000003', 'Chicago', 101),
(2004, 'Linda', 'linda@email.com', '5551000004', 'Houston', 101),
(2005, 'Richard', 'richard@email.com', '5551000005', 'Phoenix', 101);
create table Warehouse_Customer (Customer_ID int, Customer_Name varchar(100), Email varchar(100), Phone varchar(20), City varchar(100), Batch_ID int, Load_Date date);
insert into Warehouse_Customer (Customer_ID, Customer_Name, Email, Phone, City, Batch_ID, Load_Date)
values (2021, 'Mark', 'mark@email.com', '5551000021', 'Boston', 105, '2026-08-05'),
(2022, 'Betty', 'betty@email.com', '5551000022', 'Nashville', 105, '2026-08-05'),
(2023, 'Steven', 'steven@email.com', '5551000023', 'Detroit', 105, '2026-08-05'),
(2024, 'Helen', 'helen@email.com', '5551000024', 'Portland', 105, '2026-08-05'),
(2025, 'Paul', 'paul@email.com', '5551000025', 'Las Vegas', 105, '2026-08-05');
Create table Rejected_Records (
    Reject_ID int primary key,
    Customer_ID int,
    Batch_ID int,
    Reject_Reason varchar(200)
);
insert into Rejected_Records (Reject_ID, Customer_ID, Batch_ID, Reject_Reason)
values (1, 1003, 102, 'Duplicate Customer ID'),
(2, 1008, 104, 'Invalid Email Address'),
(3, 1013, 106, 'Customer Name is NULL'),
(4, 1014, 106, 'Email is NULL'),
(5, 1015, 107, 'Phone Number is NULL'),
(6, 1016, 107, 'Multiple Mandatory Fields Missing'),
(7, 1017, 108, 'Phone Number is NULL'),
(8, 1006, 103, 'Invalid Customer Data');

WITH Counts AS (
    SELECT
        (SELECT COUNT(*) FROM Source_Customer) AS Source_Count,
        (SELECT COUNT(*) FROM Warehouse_Customer) AS Warehouse_Count,
        (SELECT COUNT(*) FROM Rejected_Records) AS Rejected_Count
)
SELECT
    Source_Count,
    Warehouse_Count,
    Rejected_Count,
    Source_Count - Warehouse_Count - Rejected_Count AS Difference
FROM Counts;
```

Task 30

Find tables having the highest daily growth.

```
with TableCounts as (select B.Batch_Date,'Source_Customer',count(S.Customer_ID) as Records_Loaded from Batch_Master B
left join Source_Customer S on B.Batch_ID = S.Batch_ID group by bm.Batch_Date union all
select B.Batch_Date, 'Staging_Customer',Count(S.Customer_ID) from Batch_Master B left join Staging_Customer S on B.Batch_ID = S.Batch_ID
group by B.Batch_Date union all
select B.Batch_Date, 'Warehouse_Customer', count(W.Customer_ID)from Batch_Master B
left join Warehouse_Customer W on B.Batch_ID = W.Batch_ID
group by B.Batch_Date union all
select B.Batch_Date,'Rejected_Records',Count(R.Reject_ID) from Batch_Master B
left join Rejected_Records R on B.Batch_ID = R.Batch_ID
group by B.Batch_Date),
Growth as (select Batch_Date,Table_Name, Records_Loaded,Records_Loaded - lag(Records_Loaded) over (partition by Table_Name
order by Batch_Date) as Daily_Growth from TableCounts)
select Batch_Date, Table_Name, Records_Loaded,Daily_Growth from Growth
where Daily_Growth is not null
order by Daily_Growth desc limit 1;
-- Module 4
Create table Branch (
    Branch_ID int primary key,
    Branch_Name varchar(100),
    City varchar(100)
);
insert into Branch(Branch_ID, Branch_Name, City)
values(1, 'Chennai Branch', 'Chennai'),
(2, 'Bangalore Branch', 'Bangalore'),
(3, 'Hyderabad Branch', 'Hyderabad'),
(4, 'Mumbai Branch', 'Mumbai'),
(5, 'Delhi Branch', 'Delhi');
create table Customer (
    Customer_ID int primary key,
    Customer_Name varchar(100),
    Mobile_Number varchar(15),
    City varchar(100)
);
insert into Customer(Customer_ID, Customer_Name, Mobile_Number, City)
values(1001, 'John', '9876543210', 'Chennai'),
(1002, 'Emily', '9876543211', 'Bangalore'),
(1003, 'Michael', '9876543212', 'Hyderabad'),
(1004, 'Sophia', '9876543213', 'Mumbai'),
(1005, 'William', '9876543214', 'Delhi'),
(1006, 'Olivia', '9876543215', 'Chennai'),
(1007, 'James', '9876543216', 'Bangalore'),
(1008, 'Charlotte', '9876543217', 'Hyderabad'),
(1009, 'Benjamin', '9876543218', 'Mumbai'),
(1010, 'Amelia', '9876543219', 'Delhi');
Create table Account (Account_ID int primary key,
Customer_ID int,Branch_ID int,Account_Type varchar(30),Opening_Date date,Balance decimal(12,2),Account_Status varchar(20));
insert into Account(Account_ID, Customer_ID, Branch_ID, Account_Type, Opening_Date, Balance, Account_Status)
values(5001, 1001, 1, 'Savings', '2025-01-15', 25000.00, 'Active'),
(5002, 1002, 2, 'Current', '2025-02-20', 45000.00, 'Active'),
(5003, 1003, 3, 'Savings', '2025-03-10', 18000.00, 'Active'),
(5004, 1004, 4, 'Current', '2025-04-05', 62000.00, 'Active'),
(5005, 1005, 5, 'Savings', '2025-05-12', 32000.00, 'Active'),
(5006, 1006, 1, 'Savings', '2025-06-18', 15000.00, 'Active'),
(5007, 1007, 2, 'Current', '2025-07-22', 55000.00, 'Active'),
(5008, 1008, 3, 'Savings', '2025-08-14', 21000.00, 'Inactive'),
(5009, 1009, 4, 'Current', '2025-09-30', 75000.00, 'Active');
Create table Transactions (Transaction_ID int primary key,Account_ID int,Transaction_Date date,Transaction_Type varchar(20),
Transaction_Amount decimal(12,2));
insert into Transactions(Transaction_ID, Account_ID, Transaction_Date, Transaction_Type, Transaction_Amount)
values (9001, 5001, '2025-10-05', 'Deposit', 10000.00),
(9002, 5002, '2025-10-10', 'Withdrawal', 5000.00),
(9003, 5003, '2025-11-15', 'Deposit', 15000.00),
(9004, 5004, '2025-11-20', 'Withdrawal', 8000.00),
(9005, 5005, '2025-12-05', 'Deposit', 12000.00),
(9006, 5006, '2025-12-10', 'Withdrawal', 3000.00);
create table Transaction_Audit (Audit_ID int primary key auto_increment,Transaction_ID int,Account_ID int,Transaction_Amount decimal(12,2),
Deleted_Date datetime);
Delimiter //
create trigger trg_transaction_delete after delete on Transactions for each row
begin insert into Transaction_Audit (Transaction_ID, Account_ID, Transaction_Amount, Deleted_Date) values
(old.Transaction_ID,old.Account_ID,old.Transaction_Amount,Now());
end //
Delimiter ;

WITH TableCounts AS (
    SELECT
        b.Batch_Date,
        'Source_Customer' AS Table_Name,
        COUNT(s.Customer_ID) AS Records_Loaded
```

```

FROM Batch_Master AS b
LEFT JOIN Source_Customer AS s
    ON s.Batch_ID = b.Batch_ID
GROUP BY b.Batch_Date

UNION ALL

SELECT
    b.Batch_Date,
    'Staging_Customer',
    COUNT(s.Customer_ID)
FROM Batch_Master AS b
LEFT JOIN Staging_Customer AS s
    ON s.Batch_ID = b.Batch_ID
GROUP BY b.Batch_Date

UNION ALL

SELECT
    b.Batch_Date,
    'Warehouse_Customer',
    COUNT(w.Customer_ID)
FROM Batch_Master AS b
LEFT JOIN Warehouse_Customer AS w
    ON w.Batch_ID = b.Batch_ID
GROUP BY b.Batch_Date

UNION ALL

SELECT
    b.Batch_Date,
    'Rejected_Records',
    COUNT(r.Reject_ID)
FROM Batch_Master AS b
LEFT JOIN Rejected_Records AS r
    ON r.Batch_ID = b.Batch_ID
GROUP BY b.Batch_Date
),
Growth AS (
    SELECT
        Batch_Date,
        Table_Name,
        Records_Loaded,
        Records_Loaded - LAG(Records_Loaded) OVER (
            PARTITION BY Table_Name
            ORDER BY Batch_Date
        ) AS Daily_Growth
    FROM TableCounts
)
SELECT
    Batch_Date,
    Table_Name,
    Records_Loaded,
    Daily_Growth
FROM Growth
WHERE Daily_Growth IS NOT NULL
ORDER BY Daily_Growth DESC
LIMIT 1;

```


Task 31

Generate branch-wise total deposits.

```
Create table Branch (
    Branch_ID int primary key,
    Branch_Name varchar(100),
    City varchar(100)
);
insert into Branch(Branch_ID, Branch_Name, City)
create table Customer (
    Customer_ID int primary key,
    Customer_Name varchar(100),
    Mobile_Number varchar(15),
    City varchar(100)
);
insert into Customer(Customer_ID, Customer_Name, Mobile_Number, City)
Create table Account (Account_ID int primary key,
    Account_Type, Opening_Date, Balance, Account_Status)
Create table Transactions (Transaction_ID int primary key,Account_ID int,Transaction_Date date,Transaction_Type varchar(20),
insert into Transactions(Transaction_ID, Account_ID, Transaction_Date, Transaction_Type, Transaction_Amount)
create table Transaction_Audit (Audit_ID int primary key auto_increment,Transaction_ID int,Account_ID int,Transaction_Amount decimal(10,2))
Delimiter //
create trigger trg_transaction_delete after delete on Transactions for each row
Delimiter ;
Create a View for branch-wise transaction summary.*/
Create view Branch_Transaction_Report as
delimiter **
create procedure Get_Account_Statement(in p_Account_ID int) begin
delimiter ;
Create a Trigger that stores deleted transaction records into an audit table. */
delimiter **
create trigger trg_transaction_audit after delete on Transactions for each row
delimiter ;
DELIMITER //
Create a BEFORE INSERT Trigger to reject transactions with negative amounts.*/
delimiter **
create trigger trg_validate_transaction_amount
delimiter ;
create index idx_transactions_account_date on Transactions (Account_ID, Transaction_Date);

SELECT
    b.Branch_Name,
    SUM(t.Transaction_Amount) AS Total_Deposit_Amount
FROM Branch AS b
JOIN Account AS a
    ON a.Branch_ID = b.Branch_ID
JOIN Transactions AS t
    ON t.Account_ID = a.Account_ID
WHERE t.Transaction_Type = 'Deposit'
GROUP BY
    b.Branch_ID,
    b.Branch_Name
ORDER BY Total_Deposit_Amount DESC;
```

Task 32

Identify customers with multiple active accounts.

```
Create table Branch (
    Branch_ID int primary key,
    Branch_Name varchar(100),
    City varchar(100)
);
insert into Branch(Branch_ID, Branch_Name, City)
create table Customer (
    Customer_ID int primary key,
    Customer_Name varchar(100),
    Mobile_Number varchar(15),
    City varchar(100)
);
insert into Customer(Customer_ID, Customer_Name, Mobile_Number, City)
Create table Account (Account_ID int primary key,
    Create table Transactions (Transaction_ID int primary key,Account_ID int,Transaction_Date date,Transaction_Type varchar(20),
insert into Transactions(Transaction_ID, Account_ID, Transaction_Date, Transaction_Type, Transaction_Amount)
create table Transaction_Audit (Audit_ID int primary key auto_increment,Transaction_ID int,Account_ID int,Transaction_Amount decimal(10,2))
Delimiter //
create trigger trg_transaction_delete after delete on Transactions for each row
Delimiter ;
Create a View for branch-wise transaction summary.*/
Create view Branch_Transaction_Report as
delimiter **
create procedure Get_Account_Statement(in p_Account_ID int) begin
delimiter ;
Create a Trigger that stores deleted transaction records into an audit table. */
delimiter **
create trigger trg_transaction_audit after delete on Transactions for each row
delimiter ;
DELIMITER //
Create a BEFORE INSERT Trigger to reject transactions with negative amounts.*/
delimiter **
create trigger trg_validate_transaction_amount
delimiter ;
create index idx_transactions_account_date on Transactions (Account_ID, Transaction_Date);

SELECT
    c.Customer_ID,
    c.Customer_Name,
    COUNT(*) AS Active_Account_Count
FROM Customer AS c
JOIN Account AS a
    ON a.Customer_ID = c.Customer_ID
WHERE a.Account_Status = 'Active'
GROUP BY
    c.Customer_ID,
    c.Customer_Name
HAVING COUNT(*) > 1
ORDER BY Active_Account_Count DESC;
```

Task 33

Display the latest transaction for every account.

```
Create table Branch (
    Branch_ID int primary key,
    Branch_Name varchar(100),
    City varchar(100)
);
insert into Branch(Branch_ID, Branch_Name, City)
create table Customer (
    Customer_ID int primary key,
    Customer_Name varchar(100),
    Mobile_Number varchar(15),
    City varchar(100)
);
insert into Customer(Customer_ID, Customer_Name, Mobile_Number, City)
Create table Account (Account_ID int primary key,
    Create table Transactions (Transaction_ID int primary key,Account_ID int,Transaction_Date date,Transaction_Type varchar(20),
insert into Transactions(Transaction_ID, Account_ID, Transaction_Date, Transaction_Type, Transaction_Amount)
create table Transaction_Audit (Audit_ID int primary key auto_increment,Transaction_ID int,Account_ID int,Transaction_Amount decimal(10,2))
Delimiter //
create trigger trg_transaction_delete after delete on Transactions for each row
Delimiter ;
Create a View for branch-wise transaction summary.*/
Create view Branch_Transaction_Report as
delimiter **
create procedure Get_Account_Statement(in p_Account_ID int) begin
delimiter ;
Create a Trigger that stores deleted transaction records into an audit table. */
delimiter **
create trigger trg_transaction_audit after delete on Transactions for each row
delimiter ;
DELIMITER //
Create a BEFORE INSERT Trigger to reject transactions with negative amounts.*/
delimiter **
create trigger trg_validate_transaction_amount
delimiter ;
create index idx_transactions_account_date on Transactions (Account_ID, Transaction_Date);

SELECT
    Account_ID,
    Transaction_Date,
    Transaction_Type,
    Transaction_Amount
FROM (
    SELECT
        t.*,
        ROW_NUMBER() OVER (
            PARTITION BY Account_ID
            ORDER BY Transaction_Date DESC, Transaction_ID DESC
        ) AS rn
    FROM Transactions AS t
) AS ranked
WHERE rn = 1
ORDER BY Account_ID;
```

Task 34

Rank customers based on transaction amount.

```
Create table Branch (
    Branch_ID int primary key,
    Branch_Name varchar(100),
    City varchar(100)
);
insert into Branch(Branch_ID, Branch_Name, City)
create table Customer (
    Customer_ID int primary key,
    Customer_Name varchar(100),
    Mobile_Number varchar(15),
    City varchar(100)
);
insert into Customer(Customer_ID, Customer_Name, Mobile_Number, City)
Create table Account (Account_ID int primary key,
    insert into Account(Account_ID, Customer_ID, Branch_ID, Account_Type, Opening_Date, Balance, Account_Status)
Create table Transactions (Transaction_ID int primary key,Account_ID int,Transaction_Date date,Transaction_Type varchar(20),
insert into Transactions(Transaction_ID, Account_ID, Transaction_Date, Transaction_Type, Transaction_Amount)
create table Transaction_Audit (Audit_ID int primary key auto_increment,Transaction_ID int,Account_ID int,Transaction_Amount decimal(10,2))
Delimiter //
create trigger trg_transaction_delete after delete on Transactions for each row
Delimiter ;
Create a View for branch-wise transaction summary.*/
Create view Branch_Transaction_Report as
delimiter **
create procedure Get_Account_Statement(in p_Account_ID int) begin
delimiter ;
Create a Trigger that stores deleted transaction records into an audit table. */
delimiter **
create trigger trg_transaction_audit after delete on Transactions for each row
delimiter ;
DELIMITER //
Create a BEFORE INSERT Trigger to reject transactions with negative amounts.*/
delimiter **
create trigger trg_validate_transaction_amount
delimiter ;
create index idx_transactions_account_date on Transactions (Account_ID, Transaction_Date);

WITH CustomerTotals AS (
    SELECT
        c.Customer_ID,
        c.Customer_Name,
        SUM(t.Transaction_Amount) AS Total_Transaction_Amount
    FROM Customer AS c
    JOIN Account AS a
        ON a.Customer_ID = c.Customer_ID
    JOIN Transactions AS t
        ON t.Account_ID = a.Account_ID
    GROUP BY
        c.Customer_ID,
        c.Customer_Name
)
SELECT
    Customer_Name,
    Total_Transaction_Amount,
    DENSE_RANK() OVER (
        ORDER BY Total_Transaction_Amount DESC
    ) AS Customer_Rank
FROM CustomerTotals
ORDER BY Customer_Rank, Customer_Name;
```

Task 35

Identify suspicious transactions greater than three times the customer's average transaction amount.

```
Create table Branch (
    Branch_ID int primary key,
    Branch_Name varchar(100),
    City varchar(100)
);
insert into Branch(Branch_ID, Branch_Name, City)
create table Customer (
    Customer_ID int primary key,
    Customer_Name varchar(100),
    Mobile_Number varchar(15),
    City varchar(100)
);
insert into Customer(Customer_ID, Customer_Name, Mobile_Number, City)
Create table Account (Account_ID int primary key,
    Account_Type, Opening_Date, Balance, Account_Status)
Create table Transactions (Transaction_ID int primary key,Account_ID int,Transaction_Date date,Transaction_Type varchar(20),
insert into Transactions(Transaction_ID, Account_ID, Transaction_Date, Transaction_Type, Transaction_Amount)
create table Transaction_Audit (Audit_ID int primary key auto_increment,Transaction_ID int,Account_ID int,Transaction_Amount decimal(10,2))
Delimiter //
create trigger trg_transaction_delete after delete on Transactions for each row
Delimiter ;
Create a View for branch-wise transaction summary.*/
Create view Branch_Transaction_Report as
delimiter **
create procedure Get_Account_Statement(in p_Account_ID int) begin
delimiter ;
Create a Trigger that stores deleted transaction records into an audit table. */
delimiter **
create trigger trg_transaction_audit after delete on Transactions for each row
delimiter ;
DELIMITER //
Create a BEFORE INSERT Trigger to reject transactions with negative amounts.*/
delimiter **
create trigger trg_validate_transaction_amount
delimiter ;
create index idx_transactions_account_date on Transactions (Account_ID, Transaction_Date);

WITH CustomerAverages AS (
    SELECT
        a.Customer_ID,
        AVG(t.Transaction_Amount) AS Average_Transaction
    FROM Account AS a
    JOIN Transactions AS t
        ON t.Account_ID = a.Account_ID
    GROUP BY a.Customer_ID
)
SELECT
    c.Customer_Name,
    t.Account_ID,
    t.Transaction_Date,
    t.Transaction_Amount,
    ca.Average_Transaction AS Customer_Average_Transaction
FROM Customer AS c
JOIN CustomerAverages AS ca
    ON ca.Customer_ID = c.Customer_ID
JOIN Account AS a
    ON a.Customer_ID = c.Customer_ID
JOIN Transactions AS t
    ON t.Account_ID = a.Account_ID
WHERE t.Transaction_Amount > 3 * ca.Average_Transaction
ORDER BY t.Transaction_Amount DESC;
```

Task 36

Create a View for branch-wise transaction summary.*/

```
CREATE OR REPLACE VIEW Branch_Transaction_Report AS
SELECT
    b.Branch_ID,
    b.Branch_Name,
    COUNT(t.Transaction_ID) AS Total_Transactions,
    SUM(
        CASE
            WHEN t.Transaction_Type = 'Deposit'
            THEN t.Transaction_Amount
            ELSE 0
        END
    ) AS Total_Deposit_Amount,
    SUM(
        CASE
            WHEN t.Transaction_Type = 'Withdrawal'
            THEN t.Transaction_Amount
            ELSE 0
        END
    ) AS Total-Withdrawal_Amount
FROM Branch AS b
JOIN Account AS a
    ON a.Branch_ID = b.Branch_ID
JOIN Transactions AS t
    ON t.Account_ID = a.Account_ID
GROUP BY
    b.Branch_ID,
    b.Branch_Name;
```

Task 37

Write a Stored Procedure to generate customer account statements.

```
Create table Branch (
    Branch_ID int primary key,
    Branch_Name varchar(100),
    City varchar(100)
);
insert into Branch(Branch_ID, Branch_Name, City)
create table Customer (
    Customer_ID int primary key,
    Customer_Name varchar(100),
    Mobile_Number varchar(15),
    City varchar(100)
);
insert into Customer(Customer_ID, Customer_Name, Mobile_Number, City)
Create table Account (Account_ID int primary key,
    insert into Account(Account_ID, Customer_ID, Branch_ID, Account_Type, Opening_Date, Balance, Account_Status)
Create table Transactions (Transaction_ID int primary key,Account_ID int,Transaction_Date date,Transaction_Type varchar(20),
insert into Transactions(Transaction_ID, Account_ID, Transaction_Date, Transaction_Type, Transaction_Amount)
create table Transaction_Audit (Audit_ID int primary key auto_increment,Transaction_ID int,Account_ID int,Transaction_Amount decimal(10,2))
Delimiter //
create trigger trg_transaction_delete after delete on Transactions for each row
Delimiter ;
Create a View for branch-wise transaction summary.*/
Create view Branch_Transaction_Report as
delimiter **
create procedure Get_Account_Statement(in p_Account_ID int) begin
delimiter ;
Create a Trigger that stores deleted transaction records into an audit table. */
delimiter **
create trigger trg_transaction_audit after delete on Transactions for each row
delimiter ;
DELIMITER //
Create a BEFORE INSERT Trigger to reject transactions with negative amounts.*/
delimiter **
create trigger trg_validate_transaction_amount
before insert on Transactions
for each row
when t.Transaction_Amount < 0
begin
    insert into Transaction_Audit (Transaction_ID, Account_ID, Transaction_Amount)
    values (t.Transaction_ID, t.Account_ID, t.Transaction_Amount);
end;
delimiter ;
create index idx_transactions_account_date on Transactions (Account_ID, Transaction_Date);
DELIMITER //

CREATE PROCEDURE Get_Account_Statement(
    IN p_Account_ID INT
)
BEGIN
    SELECT
        t.Account_ID,
        t.Transaction_Date,
        t.Transaction_Type,
        t.Transaction_Amount,
        a.Balance AS Current_Balance
    FROM Transactions AS t
    JOIN Account AS a
        ON a.Account_ID = t.Account_ID
    WHERE t.Account_ID = p_Account_ID
    ORDER BY
        t.Transaction_Date,
        t.Transaction_ID;
END //

DELIMITER ;
```

Task 38

```
Create a Trigger that stores deleted transaction records into an audit table. */
DELIMITER //

CREATE TRIGGER trg_transaction_audit
AFTER DELETE ON Transactions
FOR EACH ROW
BEGIN
    INSERT INTO Transaction_Audit (
        Transaction_ID,
        Account_ID,
        Transaction_Amount,
        Deleted_Date
    )
    VALUES (
        OLD.Transaction_ID,
        OLD.Account_ID,
        OLD.Transaction_Amount,
        NOW()
    );
END //

DELIMITER ;
```


Task 39

```
Create a BEFORE INSERT Trigger to reject transactions with negative amounts.*/
DELIMITER //

CREATE TRIGGER trg_validate_transaction_amount
BEFORE INSERT ON Transactions
FOR EACH ROW
BEGIN
    IF NEW.Transaction_Amount < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Transaction amount cannot be negative';
    END IF;
END //

DELIMITER ;
```

Task 40

Recommend appropriate indexes to improve transaction search performance.

```
Create table Branch (
    Branch_ID int primary key,
    Branch_Name varchar(100),
    City varchar(100)
);
insert into Branch(Branch_ID, Branch_Name, City)
create table Customer (
    Customer_ID int primary key,
    Customer_Name varchar(100),
    Mobile_Number varchar(15),
    City varchar(100)
);
insert into Customer(Customer_ID, Customer_Name, Mobile_Number, City)
Create table Account (Account_ID int primary key,
    Customer_ID int, Branch_ID, Account_Type, Opening_Date, Balance, Account_Status)
Create table Transactions (Transaction_ID int primary key, Account_ID int, Transaction_Date date, Transaction_Type varchar(20),
    Transaction_Amount decimal(10,2))
insert into Transactions(Transaction_ID, Account_ID, Transaction_Date, Transaction_Type, Transaction_Amount)
create table Transaction_Audit (Audit_ID int primary key auto_increment, Transaction_ID int, Account_ID int, Transaction_Amount decimal(10,2))
Delimiter //
create trigger trg_transaction_delete after delete on Transactions for each row
Delimiter ;
Create a View for branch-wise transaction summary.*/
Create view Branch_Transaction_Report as
delimiter **
create procedure Get_Account_Statement(in p_Account_ID int) begin
delimiter ;
Create a Trigger that stores deleted transaction records into an audit table. */
delimiter **
create trigger trg_transaction_audit after delete on Transactions for each row
delimiter ;
DELIMITER //
Create a BEFORE INSERT Trigger to reject transactions with negative amounts.*/
delimiter **
create trigger trg_validate_transaction_amount
before insert on Transactions
for each row
when (Transaction_Amount < 0)
rollback;
delimiter ;
create index idx_transactions_account_date on Transactions (Account_ID, Transaction_Date);

CREATE INDEX idx_transactions_account_date
ON Transactions (Account_ID, Transaction_Date);

SELECT
    Transaction_ID,
    Account_ID,
    Transaction_Date,
    Transaction_Type,
    Transaction_Amount
FROM Transactions
WHERE Account_ID = 5001
AND Transaction_Date = '2025-10-05';
```

Task 41

Using a CTE, generate department-wise average salary and display employees earning above that average.

```
create table Department (
    Department_ID int primary key,
    Department_Name varchar(100)
);
insert into Department(Department_ID, Department_Name)
create table Employee (
insert into Employee
create table Employee_History (
    Employee_ID int,
    Employee_Name varchar(100),
    Department_ID int,
    Salary decimal(10,2),
    Effective_Date date
);
insert into Employee_History
create table Category (
    Category_ID int primary key,
    Category_Name varchar(100)
);
insert into Category(Category_ID, Category_Name)
Create table Product (
    Product_ID int primary key,
    Product_Name VARCHAR(100),
    Category_ID INT,
    Unit_Price DECIMAL(10,2)
);
insert into Product(Product_ID, Product_Name, Category_ID, Unit_Price)
create table Customer (
    Customer_ID int primary key,
    Customer_Name varchar(100),
    Region varchar(50)
);
insert into Customer(Customer_ID, Customer_Name, Region)
create table Orders (
    Order_ID int primary Key,
    Customer_ID int,
    Order_Date date
);
insert into Orders(Order_ID, Customer_ID, Order_Date)
create table Fact_Sales (
    Sale_ID int primary key,
    Order_ID int,
    Product_ID int,
    Quantity int,
    Sales_Amount decimal(12,2),
    Batch_ID int
);
insert into Fact_Sales(Sale_ID, Order_ID, Product_ID, Quantity, Sales_Amount, Batch_ID)
create table Staging_Sales (Sale_ID int primary Key, Order_ID int, Product_ID int, Quantity int, Sales_Amount decimal(12,2),
insert into Staging_Sales(Sale_ID, Order_ID, Product_ID, Quantity, Sales_Amount, Validation_Status, Batch_ID)
create table ETL_Batch (
    Batch_ID int primary key,
    Batch_Date DATE,
    Source_Count int,
    Staging_Count int,
    Warehouse_Count int
);
insert into ETL_Batch(Batch_ID, Batch_Date, Source_Count, Staging_Count, Warehouse_Count)
Create table Fact_Sales_Audit (
    Audit_ID int primary key auto_increment,
    Sale_ID int,
    Action_Type varchar(20),
    Action_Date datetime
);
Create a Temporary Table for the top-selling products and generate multiple reports from it. */
Create temporary table Top_Selling_Products as
Create a Stored Procedure to load validated records from the staging table into the Data
delimiter **
create procedure Load_Valid_Sales()
delimiter ;
Create a Trigger that logs every INSERT, UPDATE, and DELETE performed on the Fact_Sales
delimiter //
create trigger trg_fact_sales_after_insert after insert on Fact_Sales for each row
insert into Fact_Sales_Audit (Sale_ID, Action_Type, Action_Date)
delimiter ; -- just change insert to delete or update
Create a View that combines Customer, Orders, and Sales information for business reporting. */
Create view Customer_Order_Sales_Report as
```

```

Create index idx_orders_customer on Orders (Customer_ID);

WITH DepartmentAverage AS (
  SELECT
    Department_ID,
    AVG(Salary) AS Average_Salary
  FROM Employee
  GROUP BY Department_ID
)
SELECT
  d.Department_Name,
  e.Employee_Name,
  e.Salary,
  da.Average_Salary
FROM Employee AS e
JOIN DepartmentAverage AS da
  ON da.Department_ID = e.Department_ID
JOIN Department AS d
  ON d.Department_ID = e.Department_ID
WHERE e.Salary > da.Average_Salary
ORDER BY
  d.Department_Name,
  e.Salary DESC;

```

Task 42

Generate the top five products for every category using Window Functions.

```
with Product_Sales as (
select c.Category_ID,c.Category_Name,p.Product_ID,p.Product_Name,sum(fs.Sales_Amount) as Total_Sales from Category c
inner join Product on c.Category_ID = p.Category_ID
inner join Fact_Sales fs on p.Product_ID = fs.Product_ID
group by c.Category_ID,c.Category_Name,p.Product_ID,p.Product_Name),
Ranked_Products as (
select Category_Name,Product_Name,Total_Sales,
dense_rank() over (partition by Category_ID order by Total_Sales desc) as Product_Rank from Product_Sales)
select Category_Name,Product_Name,Total_Sales,Product_Rank AS "Rank"from Ranked_Products
where Product_Rank <= 5
order by Category_Name,Product_Rank;

WITH ProductSales AS (
    SELECT
        c.Category_ID,
        c.Category_Name,
        p.Product_ID,
        p.Product_Name,
        SUM(fs.Sales_Amount) AS Total_Sales
    FROM Category AS c
    JOIN Product AS p
        ON p.Category_ID = c.Category_ID
    JOIN Fact_Sales AS fs
        ON fs.Product_ID = p.Product_ID
    GROUP BY
        c.Category_ID,
        c.Category_Name,
        p.Product_ID,
        p.Product_Name
),
RankedProducts AS (
    SELECT
        ProductSales.*,
        ROW_NUMBER() OVER (
            PARTITION BY Category_ID
            ORDER BY Total_Sales DESC, Product_ID
        ) AS Product_Rank
    FROM ProductSales
)
SELECT
    Category_Name,
    Product_Name,
    Total_Sales,
    Product_Rank AS Rank
FROM RankedProducts
WHERE Product_Rank <= 5
ORDER BY
    Category_Name,
    Product_Rank;
```

Task 43

Create a Temporary Table for the top-selling products and generate multiple reports from it. */

Create temporary table Top_Selling_Products as

```
select p.Product_ID,p.Product_Name,sum(fs.Sales_Amount) as Total_Sales from Product p
inner join Fact_Sales fs on p.Product_ID = fs.Product_ID
group by p.Product_ID, p.Product_Name
order by Total_Sales desc
limit 5;
```

CREATE TEMPORARY TABLE Top_Selling_Products AS
SELECT

```
    p.Product_ID,
    p.Product_Name,
    SUM(fs.Sales_Amount) AS Total_Sales
FROM Product AS p
JOIN Fact_Sales AS fs
    ON fs.Product_ID = p.Product_ID
GROUP BY
    p.Product_ID,
    p.Product_Name
ORDER BY Total_Sales DESC
LIMIT 5;
```

```
SELECT
    Product_ID,
    Product_Name,
    Total_Sales
FROM Top_Selling_Products
ORDER BY Total_Sales DESC;
```

```
SELECT
    COUNT(*) AS Top_Product_Count,
    SUM(Total_Sales) AS Top_Five_Revenue,
    AVG(Total_Sales) AS Average_Top_Product_Sales
FROM Top_Selling_Products;
```

Task 44

Create a Stored Procedure to load validated records from the staging table into the Data Warehouse. */

delimiter **

create procedure Load_Valid_Sales()

begin insert into Fact_Sales(Sale_ID,Order_ID,Product_ID,Quantity,Sales_Amount,Batch_ID)

select Sale_ID,Order_ID,Product_ID,Quantity,Sales_Amount,Batch_ID from Staging_Sales

where Validation_Status = 'Valid';

end **

delimiter ;

DELIMITER //

CREATE PROCEDURE Load_Valid_Sales()

BEGIN

INSERT INTO Fact_Sales (

Sale_ID,

Order_ID,

Product_ID,

Quantity,

Sales_Amount,

Batch_ID

)

SELECT

Sale_ID,

Order_ID,

Product_ID,

Quantity,

Sales_Amount,

Batch_ID

FROM Staging_Sales

WHERE Validation_Status = 'Valid';

END //

DELIMITER ;

Task 45

```
Create a Trigger that logs every INSERT, UPDATE, and DELETE performed on the Fact_Sales
table. */
delimiter //
create trigger trg_fact_sales_after_insert after insert on Fact_Sales for each row
begin
insert into Fact_Sales_Audit (Sale_ID, Action_Type, Action_Date)
values(new.Sale_ID, 'insert', Now());
end //
delimiter ; -- just change insert to delete or update

DELIMITER //

CREATE TRIGGER trg_fact_sales_insert
AFTER INSERT ON Fact_Sales
FOR EACH ROW
BEGIN
    INSERT INTO Fact_Sales_Audit (
        Sale_ID,
        Action_Type,
        Action_Date
    )
    VALUES (
        NEW.Sale_ID,
        'INSERT',
        NOW()
    );
END //

CREATE TRIGGER trg_fact_sales_update
AFTER UPDATE ON Fact_Sales
FOR EACH ROW
BEGIN
    INSERT INTO Fact_Sales_Audit (
        Sale_ID,
        Action_Type,
        Action_Date
    )
    VALUES (
        NEW.Sale_ID,
        'UPDATE',
        NOW()
    );
END //

CREATE TRIGGER trg_fact_sales_delete
AFTER DELETE ON Fact_Sales
FOR EACH ROW
BEGIN
    INSERT INTO Fact_Sales_Audit (
        Sale_ID,
        Action_Type,
        Action_Date
    )
    VALUES (
        OLD.Sale_ID,
        'DELETE',
        NOW()
    );
END //

DELIMITER ;
```


Task 46

Create a View that combines Customer, Orders, and Sales information for business reporting. */

```
CREATE OR REPLACE VIEW Customer_Order_Sales_Report AS
SELECT
    c.Customer_ID,
    c.Customer_Name,
    o.Order_ID,
    o.Order_Date,
    fs.Product_ID,
    fs.Quantity,
    fs.Sales_Amount
FROM Customer AS c
JOIN Orders AS o
    ON o.Customer_ID = c.Customer_ID
JOIN Fact_Sales AS fs
    ON fs.Order_ID = o.Order_ID;
```

Task 47

Recommend indexes for improving performance of a monthly dashboard query involving multiple joins and aggregations.

```
create table Department (
    Department_ID int primary key,
    Department_Name varchar(100)
);
insert into Department(Department_ID, Department_Name)
create table Employee (
insert into Employee
create table Employee_History (
    Employee_ID int,
    Employee_Name varchar(100),
    Department_ID int,
    Salary decimal(10,2),
    Effective_Date date
);
insert into Employee_History
create table Category (
    Category_ID int primary key,
    Category_Name varchar(100)
);
insert into Category(Category_ID, Category_Name)
Create table Product (
    Product_ID int primary key,
    Product_Name VARCHAR(100),
    Category_ID INT,
    Unit_Price DECIMAL(10,2)
);
insert into Product(Product_ID, Product_Name, Category_ID, Unit_Price)
create table Customer (
    Customer_ID int primary key,
    Customer_Name varchar(100),
    Region varchar(50)
);
insert into Customer(Customer_ID, Customer_Name, Region)
create table Orders (
    Order_ID int primary Key,
    Customer_ID int,
    Order_Date date
);
insert into Orders(Order_ID, Customer_ID, Order_Date)
create table Fact_Sales (
    Sale_ID int primary key,
    Order_ID int,
    Product_ID int,
    Quantity int,
    Sales_Amount decimal(12,2),
    Batch_ID int
);
insert into Fact_Sales(Sale_ID, Order_ID, Product_ID, Quantity, Sales_Amount, Batch_ID)
create table Staging_Sales (Sale_ID int primary Key,Order_ID int,Product_ID int,Quantity int,Sales_Amount decimal(12,2),
insert into Staging_Sales(Sale_ID, Order_ID, Product_ID, Quantity, Sales_Amount, Validation_Status, Batch_ID)
create table ETL_Batch (
    Batch_ID int primary key,
    Batch_Date DATE,
    Source_Count int,
    Staging_Count int,
    Warehouse_Count int
);
insert into ETL_Batch(Batch_ID, Batch_Date, Source_Count, Staging_Count, Warehouse_Count)
Create table Fact_Sales_Audit (
    Audit_ID int primary key auto_increment,
    Sale_ID int,
    Action_Type varchar(20),
    Action_Date datetime
);
Create a Temporary Table for the top-selling products and generate multiple reports from it. */
Create temporary table Top_Selling_Products as
Create a Stored Procedure to load validated records from the staging table into the Data
delimiter **
create procedure Load_Valid_Sales()
delimiter ;
Create a Trigger that logs every INSERT, UPDATE, and DELETE performed on the Fact_Sales
delimiter //
create trigger trg_fact_sales_after_insert after insert on Fact_Sales for each row
insert into Fact_Sales_Audit (Sale_ID, Action_Type, Action_Date)
delimiter ; -- just change insert to delete or update
Create a View that combines Customer, Orders, and Sales information for business reporting. */
Create view Customer_Order_Sales_Report as
```

```
Create index idx_orders_customer on Orders (Customer_ID);

CREATE INDEX idx_orders_customer_date
  ON Orders (Customer_ID, Order_Date);

CREATE INDEX idx_fact_sales_order_product
  ON Fact_Sales (Order_ID, Product_ID);

CREATE INDEX idx_product_category
  ON Product (Category_ID);

CREATE INDEX idx_customer_region
  ON Customer (Region);
```

Task 48

Generate a report comparing source, staging, and warehouse record counts for every batch.

```
create table Department (
    Department_ID int primary key,
    Department_Name varchar(100)
);
insert into Department(Department_ID, Department_Name)
create table Employee (
insert into Employee
create table Employee_History (
    Employee_ID int,
    Employee_Name varchar(100),
    Department_ID int,
    Salary decimal(10,2),
    Effective_Date date
);
insert into Employee_History
create table Category (
    Category_ID int primary key,
    Category_Name varchar(100)
);
insert into Category(Category_ID, Category_Name)
Create table Product (
    Product_ID int primary key,
    Product_Name VARCHAR(100),
    Category_ID INT,
    Unit_Price DECIMAL(10,2)
);
insert into Product(Product_ID, Product_Name, Category_ID, Unit_Price)
create table Customer (
    Customer_ID int primary key,
    Customer_Name varchar(100),
    Region varchar(50)
);
insert into Customer(Customer_ID, Customer_Name, Region)
create table Orders (
    Order_ID int primary Key,
    Customer_ID int,
    Order_Date date
);
insert into Orders(Order_ID, Customer_ID, Order_Date)
create table Fact_Sales (
    Sale_ID int primary key,
    Order_ID int,
    Product_ID int,
    Quantity int,
    Sales_Amount decimal(12,2),
    Batch_ID int
);
insert into Fact_Sales(Sale_ID, Order_ID, Product_ID, Quantity, Sales_Amount, Batch_ID)
create table Staging_Sales (Sale_ID int primary Key,Order_ID int,Product_ID int,Quantity int,Sales_Amount decimal(12,2),
insert into Staging_Sales(Sale_ID, Order_ID, Product_ID, Quantity, Sales_Amount, Validation_Status, Batch_ID)
create table ETL_Batch (
    Batch_ID int primary key,
    Batch_Date DATE,
    Source_Count int,
    Staging_Count int,
    Warehouse_Count int
);
insert into ETL_Batch(Batch_ID, Batch_Date, Source_Count, Staging_Count, Warehouse_Count)
Create table Fact_Sales_Audit (
    Audit_ID int primary key auto_increment,
    Sale_ID int,
    Action_Type varchar(20),
    Action_Date datetime
);
Create a Temporary Table for the top-selling products and generate multiple reports from it. */
Create temporary table Top_Selling_Products as
Create a Stored Procedure to load validated records from the staging table into the Data
delimiter **
create procedure Load_Valid_Sales()
delimiter ;
Create a Trigger that logs every INSERT, UPDATE, and DELETE performed on the Fact_Sales
delimiter //
create trigger trg_fact_sales_after_insert after insert on Fact_Sales for each row
insert into Fact_Sales_Audit (Sale_ID, Action_Type, Action_Date)
delimiter ; -- just change insert to delete or update
Create a View that combines Customer, Orders, and Sales information for business reporting. */
Create view Customer_Order_Sales_Report as
Create index idx_orders_customer on Orders (Customer_ID);
```

```
SELECT
    Batch_ID,
    Source_Count,
    Staging_Count,
    Warehouse_Count,
    Source_Count - Warehouse_Count AS Difference,
    CASE
        WHEN Source_Count = Staging_Count
            AND Staging_Count = Warehouse_Count
            THEN 'Successful Load'
        WHEN Source_Count > Warehouse_Count
            THEN 'Partially Successful'
        ELSE 'Check Required'
    END AS Load_Status
FROM ETL_Batch
ORDER BY Batch_ID;
```

Task 49

Identify slowly changing records by comparing historical employee data with the latest snapshot.

```
with Latest_History as (
select Employee_ID,Employee_Name,Department_ID,Salary,
row_number() over (partition by Employee_ID order by Effective_Date desc) as rn from Employee_History)
select e.Employee_ID,e.Employee_Name, h.Salary AS "Old Salary",e.Salary AS "New Salary",
old_d.Department_Name AS "Old Department", new_d.Department_Name AS "New Department" from Employee e
inner join Latest_History h on e.Employee_ID = h.Employee_ID and h.rn = 1
inner join Department old_d on h.Department_ID = old_d.Department_ID
inner join Department new_d on e.Department_ID = new_d.Department_ID
where e.Salary <> h.Salary or e.Department_ID <> h.Department_ID
order by e.Employee_ID;

WITH LatestHistory AS (
    SELECT
        Employee_ID,
        Employee_Name,
        Department_ID,
        Salary,
        ROW_NUMBER() OVER (
            PARTITION BY Employee_ID
            ORDER BY Effective_Date DESC
        ) AS rn
    FROM Employee_History
)
SELECT
    e.Employee_ID,
    e.Employee_Name,
    h.Salary AS Old_Salary,
    e.Salary AS New_Salary,
    old_d.Department_Name AS Old_Department,
    new_d.Department_Name AS New_Department
FROM Employee AS e
JOIN LatestHistory AS h
    ON h.Employee_ID = e.Employee_ID
   AND h.rn = 1
JOIN Department AS old_d
    ON old_d.Department_ID = h.Department_ID
JOIN Department AS new_d
    ON new_d.Department_ID = e.Department_ID
WHERE e.Salary <> h.Salary
   OR e.Department_ID <> h.Department_ID
ORDER BY e.Employee_ID;
```

Task 50

A retail company has loaded data from multiple source systems into a centralized Data

```
with Quality_Checks as (select 'null Customer Records' as Check_Name, Count(*) AS Issue_Count from Customer
where Customer_ID is null or Customer_Name is null or Region is null
union all
select 'Invalid Staging Records', count(*)from Staging_Sales
where Validation_Status = 'Invalid'
union all
select'Duplicate Staging Records',count(*)from(select Order_ID, Product_ID from Staging_Sales
group by Order_ID, Product_ID
having count(*) > 1) d
union all
select 'Negative Quantity',count(*)from Staging_Sales
where Quantity <= 0 union all
select'Negative Sales Amount',count(*)from Staging_Sales
where Sales_Amount < 0 union all
select 'Orphan Orders', count(*) from Orders o left join Customer c
on o.Customer_ID = c.Customer_ID
where c.Customer_ID is null union all
select 'Orphan Products', count(*) from Fact_Sales fs left join Product p
on fs.Product_ID = p.Product_ID
where p.Product_ID is null union all
select'Orphan Sales Orders',count(*)from Fact_Sales fs left join Orders o
on fs.Order_ID = o.Order_ID
where o.Order_ID is null union all
select 'Incorrect Sales Amount',count(*)from Fact_Sales fs inner join Product p
on fs.Product_ID = p.Product_ID
where fs.Sales_Amount <> fs.Quantity * p.Unit_Price)
select Check_Name, Issue_Count, case
when Issue_Count = 0 then 'PASS' else 'FAIL'
end from Quality_Checks
order by case
when Issue_count > 0 then 1 else 2 end, Check_Name;

WITH QualityChecks AS (
    SELECT
        'Null Customer Records' AS Check_Name,
        COUNT(*) AS Issue_Count
    FROM Customer
    WHERE Customer_ID IS NULL
        OR Customer_Name IS NULL
        OR Region IS NULL

    UNION ALL

    SELECT
        'Invalid Staging Records',
        COUNT(*)
    FROM Staging_Sales
    WHERE Validation_Status = 'Invalid'

    UNION ALL

    SELECT
        'Duplicate Staging Records',
        COUNT(*)
    FROM (
        SELECT
            Order_ID,
            Product_ID
        FROM Staging_Sales
        GROUP BY
            Order_ID,
            Product_ID
        HAVING COUNT(*) > 1
    ) AS d

    UNION ALL

    SELECT
        'Negative Quantity',
        COUNT(*)
    FROM Staging_Sales
    WHERE Quantity <= 0

    UNION ALL

    SELECT
        'Negative Sales Amount',
        COUNT(*)
    FROM Staging_Sales
```

```

WHERE Sales_Amount < 0

UNION ALL

SELECT
    'Orphan Orders',
    COUNT(*)
FROM Orders AS o
LEFT JOIN Customer AS c
    ON c.Customer_ID = o.Customer_ID
WHERE c.Customer_ID IS NULL

UNION ALL

SELECT
    'Orphan Products',
    COUNT(*)
FROM Fact_Sales AS fs
LEFT JOIN Product AS p
    ON p.Product_ID = fs.Product_ID
WHERE p.Product_ID IS NULL

UNION ALL

SELECT
    'Orphan Sales Orders',
    COUNT(*)
FROM Fact_Sales AS fs
LEFT JOIN Orders AS o
    ON o.Order_ID = fs.Order_ID
WHERE o.Order_ID IS NULL

UNION ALL

SELECT
    'Incorrect Sales Amount',
    COUNT(*)
FROM Fact_Sales AS fs
JOIN Product AS p
    ON p.Product_ID = fs.Product_ID
WHERE fs.Sales_Amount <> fs.Quantity * p.Unit_Price
)
SELECT
    Check_Name,
    Issue_Count,
    CASE
        WHEN Issue_Count = 0 THEN 'PASS'
        ELSE 'FAIL'
    END AS Result
FROM QualityChecks
ORDER BY
    CASE
        WHEN Issue_Count > 0 THEN 1
        ELSE 2
    END,
    Check_Name;

```